

UNIVERSIDAD PERUANA UNIÓN

Facultad de Ingeniería y Arquitectura
Escuela Profesional de Ingeniería de Sistemas — Sede Juliaca

GUÍA COMPLETA DE EJECUCIÓN

Evaluación Práctica Final — Unidad IV

Monitoreo de Seguridad, SIEM e Inteligencia Artificial

Curso: Seguridad Informática — Ciclo IX
Modalidad: Laboratorio Local (VirtualBox) — 2 máquinas virtuales

Presentado por:

Fran Franklin Calizaya Apaza

Repositorio: github.com/franlizaya266-sudo/seguridad

Juliaca, Perú — 2026

Cómo usar esta guía

Esta guía contiene, para cada tarea, el comando o código a ejecutar, el objetivo de la tarea, y un recuadro de “Resultado obtenido” más la referencia del screenshot exigido por la rúbrica. Los recuadros de resultado y de captura quedan marcados como [Completar] porque dependen de la ejecución real en tu propia máquina virtual: nadie puede generarlos por ti, ya que la evaluación exige evidencia de ejecución real con tu nombre y la fecha/hora visibles.

El código completo de cada script, regla de Wazuh y notebook está incluido íntegro en esta guía y además se entrega como archivos sueltos listos para copiar directamente a tu repositorio (analizar_ssh.py, analizar_web.py, visualizar.py, local_rules_ssh.xml, local_rules_exfil.xml, deteccion_anomalias.ipynb, predecir.py, README.md). Solo necesitas: 1) crear las VMs, 2) instalar el entorno, 3) copiar estos archivos a las carpetas correspondientes, 4) ejecutar cada paso y capturar la evidencia.

Convención de nombres de repositorio: se usó seguridad (apellido paterno del estudiante, en minúsculas), reemplazando cualquier nombre previo usado en borradores anteriores.

1. Introducción y Objetivos

El presente informe documenta, de manera detallada y secuencial, el desarrollo de la Evaluación Práctica Final de la Unidad IV del curso Seguridad Informática, que comprende cuatro laboratorios integrados: análisis forense de logs con Python, creación de reglas de correlación en Wazuh, desarrollo de un modelo de detección de anomalías con Machine Learning, y construcción de un dashboard de monitoreo SOC.

El objetivo general es demostrar la capacidad de diseñar e implementar un sistema de monitoreo de seguridad utilizando herramientas SIEM y técnicas de análisis de eventos en tiempo real, integrando inteligencia artificial para la detección de anomalías y respondiendo proactivamente a incidentes en entornos empresariales.

Objetivos específicos

- Analizar logs de autenticación SSH y de acceso web Apache mediante scripts en Python para identificar patrones de ataque.
- Implementar Wazuh como plataforma SIEM y configurar reglas de correlación personalizadas para detección de fuerza bruta y exfiltración de datos.
- Entrenar y evaluar un modelo de Isolation Forest para la detección no supervisada de anomalías en tráfico de red.
- Construir un dashboard de monitoreo SOC que integre visualizaciones en tiempo real de las alertas generadas por Wazuh.
- Documentar el proceso completo de configuración del entorno y las evidencias de ejecución real.

Repositorio del curso (datasets e indicaciones base):

github.com/abelthf/examen_final_seguridad_informatica

2. Arquitectura del Laboratorio y Configuración de Máquinas Virtuales

Se optó por un entorno 100% local sobre Oracle VirtualBox, compuesto por dos máquinas virtuales con roles diferenciados. Se prescinde de una tercera VM tipo “endpoint víctima” porque los datasets necesarios para los Laboratorios 1 y 3 (auth.log, access.log y network_traffic.csv) son provistos

directamente por el repositorio del curso, garantizando resultados comparables entre todos los estudiantes.

2.1 Resumen de máquinas virtuales

VM 1 — SIEM	Wazuh-SIEM · Ubuntu Server 24.04 LTS · IP 192.168.56.10 · 8–12 GB RAM · 4 vCPU · 80 GB disco
VM 2 — Atacante	Kali-Atacante · Kali Linux 2024.x · IP 192.168.56.30 · 4 GB RAM · 2 vCPU · 30 GB disco

2.2 Propósito de cada VM

Wazuh-SIEM (192.168.56.10): máquina central del laboratorio. Aloja el Wazuh Manager, Wazuh Indexer y Wazuh Dashboard (modo all-in-one), además del entorno Python/Jupyter para los Laboratorios 1 y 3. El propio Wazuh Dashboard (basado en OpenSearch Dashboards) se usa también como herramienta de visualización del Laboratorio 4, sin necesidad de instalar Kibana o Grafana por separado. En esta VM no se ataca ni se es atacado directamente: se observa, analiza y correlaciona.

Kali-Atacante (192.168.56.30): máquina ofensiva opcional, sin agente Wazuh instalado. La evidencia oficial del Laboratorio 2 se genera ejecutando el script `simular_bruteforce.sh` directamente en Wazuh-SIEM (inyecta eventos de fuerza bruta vía `syslog` local, sin necesidad de tráfico de red real). Kali-Atacante se mantiene disponible para, opcionalmente, lanzar un ataque SSH real de extremo a extremo y enriquecer con tráfico genuino el dashboard del Laboratorio 4.

2.3 Configuración de red

- Ambas VMs se configuran en una misma Red Interna / NAT Network de VirtualBox, rango 10.0.2.0/24.
- El Wazuh Dashboard (puerto 443) no se expone a internet, solo accesible dentro de la red interna y desde el equipo anfitrión.
- Se verifica conectividad entre ambas VMs mediante ping cruzado antes de iniciar los laboratorios.

2.4 Creación de la VM Wazuh-SIEM

EJECUTAR EN · Anfitrión — VirtualBox Manager

1. Nombre: Wazuh-SIEM | Tipo: Linux | Versión: Ubuntu (64-bit)

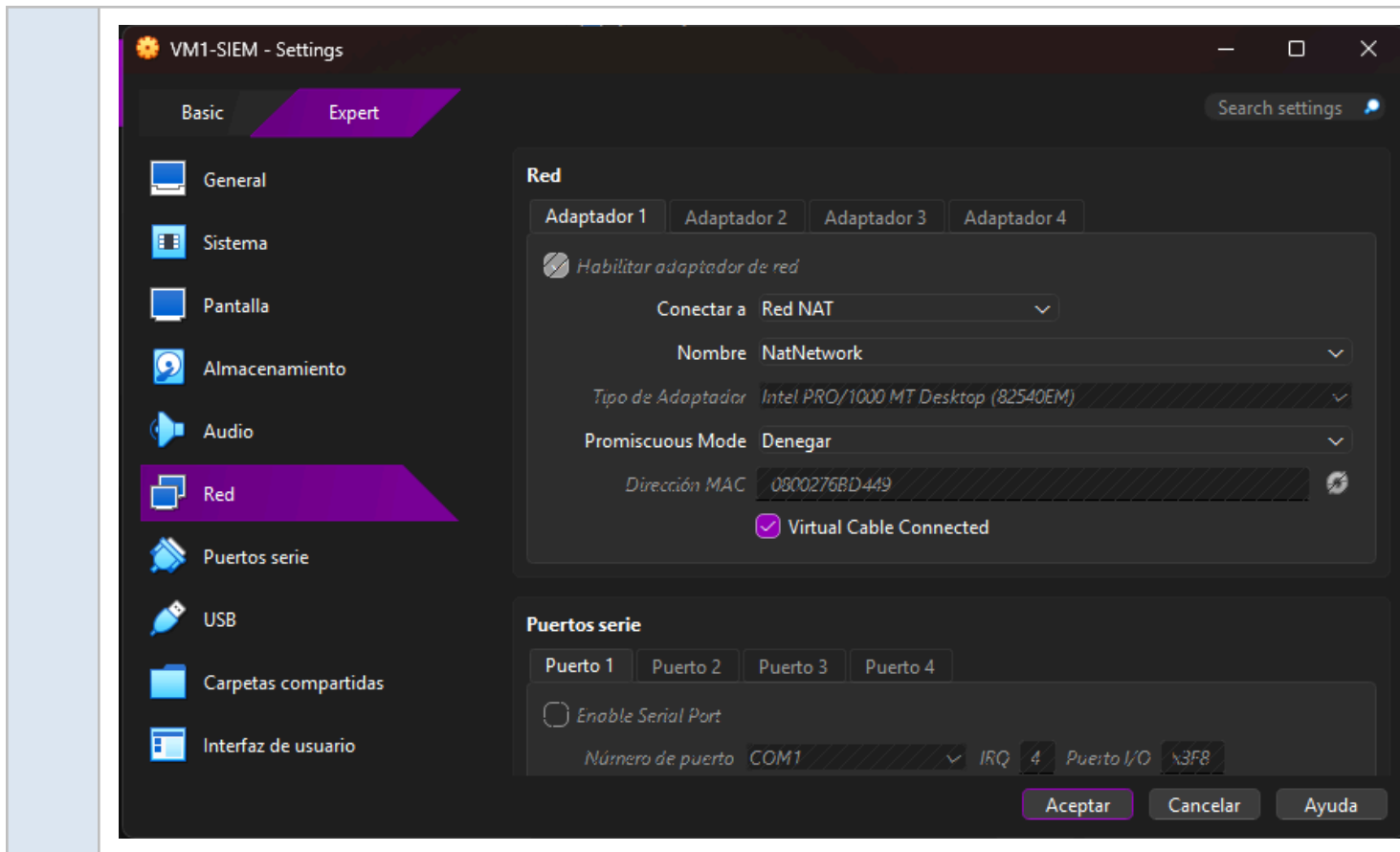
2. Memoria: 8192–12288 MB | Procesadores: 4 vCPU

3. Disco: 80 GB (VDI, reservado dinámicamente)

4. Red: Adaptador 1 en NAT Network / Red interna (192.168.56.0/24)

SCR-VM1

Configuración de la VM Wazuh-SIEM en VirtualBox (pestañas General y Sistema).

**EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24**

```
sudo apt update
sudo apt upgrade -y
sudo reboot
```

Objetivo: Actualizar todos los paquetes del sistema base antes de instalar Wazuh, evitando conflictos de dependencias.

Resultado obtenido:

```

ERROR: Invalid operation upgrade
fran@wazzu:~$ sudo apt upgrade -y
Upgrading:
  bpftool                                libxml2-16
  build-essential                        linux-generic-hwe-24.04
  cups-filters                          linux-generic-hwe-26.04
  cups-filters-core-drivers            linux-headers-generic-hwe-26.04
  curl                                 linux-image-generic-hwe-26.04
  gir1.2-packagekitglib-1.0           linux-libc-dev
  gstreamer1.0-packagekit             linux-perf
  heif-gdk-pixbuf                     linux-tools-common
  heif-thumbnailer                    localesearch
  hplip                               packagekit
  hplip-data                          perl
  libcurl3t64-gnutls                 perl-base
  libcurl4t64                        perl-modules-5.40
  libheif-plugin-aomdec               printer-driver-hpcups
  libheif-plugin-aomenc               printer-driver-postscript-hp
  libheif1                           python3-distupgrade
  libhpmud0                          tar
  libnfs14                           ubuntu-release-upgrader-core
  libnss3                             ubuntu-release-upgrader-gtk
  libpackagekit-glib2-18             update-notifier
  libperl5.40                        update-notifier-common
  libsane-hpaio                      vim-common
  libsqlite3-0                       vim-tiny
  libssh2-1t64                       xxd

Installing dependencies:
  libcrypt-dev                      linux-main-modules-zfs-7.0.0-27-generic

```

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
ip addr
```

Objetivo: Identificar la IP real asignada a la VM para documentarla y usarla en la configuración de red y en los ataques desde Kali.

Resultado obtenido:

10.0.2.3/24



Salida de 'ip addr' mostrando la dirección IP de Wazuh-SIEM.

```

fran@wazzu:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d3:77:8a brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.3/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3
        valid_lft 464sec preferred_lft 464sec
    inet6 fe80::a00:27ff:fed3:778a/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:88:be:43 brd ff:ff:ff:ff:ff:ff
    altname enx08002788be43
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute enp0s8
        valid_lft 85967sec preferred_lft 85967sec
    inet6 fd17:625c:f037:3:5725:dd0b:2203:e1ba/64 scope global dynamic mngtmpdr n noprefixroute
        valid_lft 86094sec preferred_lft 14094sec
    inet6 fd17:625c:f037:3:96e5:73e8:3faf:8819/64 scope global temporary dynamic
        valid_lft 86094sec preferred_lft 14094sec
    inet6 fe80::eca5:6049:47e2:bd0e/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
fran@wazzu:~$

```

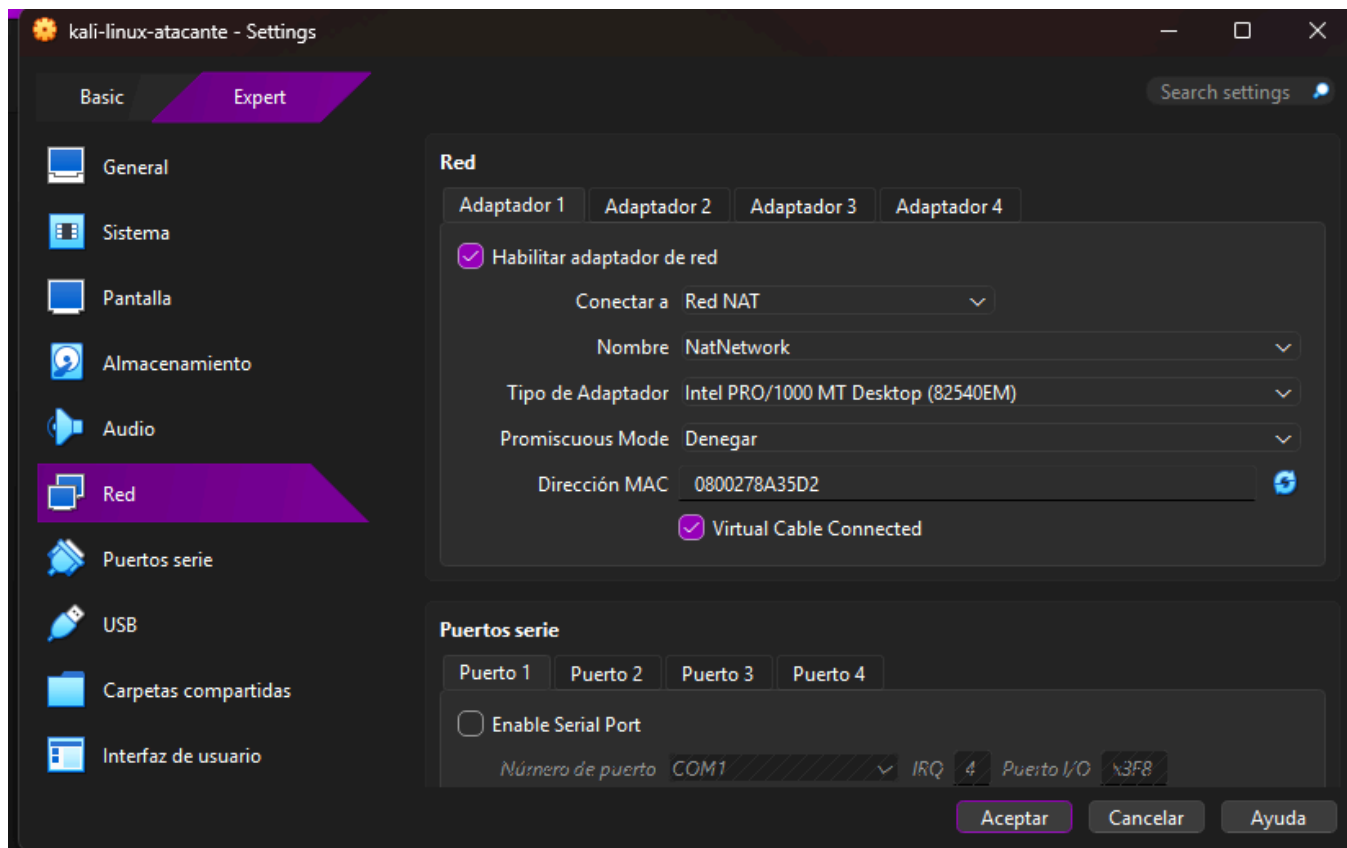
2.5 Creación de la VM Kali-Atacante

EJECUTAR EN · Anfitrión — VirtualBox Manager

1. Nombre: Kali-Atacante | Tipo: Linux | Versión: Debian (64-bit)
2. Memoria: 4096 MB | Procesadores: 2 vCPU
3. Disco: 30 GB
4. Red: Adaptador 1 en NAT Network / Red interna (192.168.56.0/24), misma red que Wazuh-SIEM



Configuración de la VM Kali-Atacante en VirtualBox.

**EJECUTAR EN · Kali-Atacante — 10.0.2.4/24**

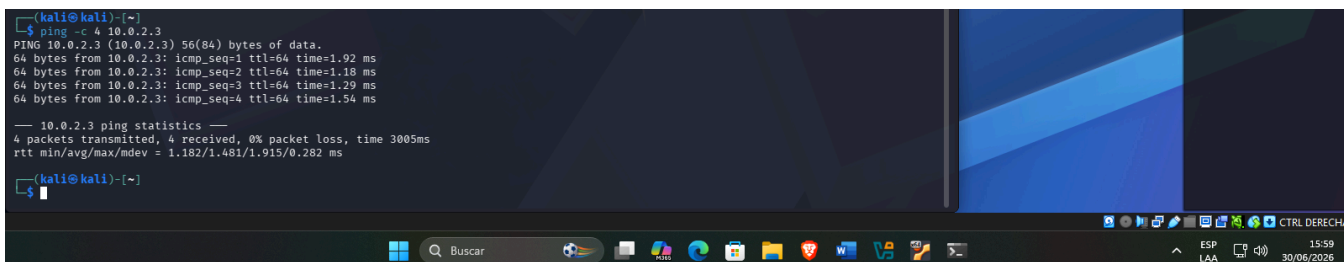
```
sudo apt update && sudo apt upgrade -y  
ip addr  
ping -c 4 10.0.2.3
```

Objetivo: Actualizar Kali, confirmar su propia IP y validar conectividad de red hacia la VM Wazuh-SIEM antes de lanzar cualquier ataque.

Resultado obtenido:

Ping exitoso desde Kali-Atacante hacia Wazuh-SIEM (10.0.2.3).

SCR-
VM4



2.6 Preparación del repositorio Git

El estudiante debe crear su propio repositorio con el nombre seguridad (apellido, en minúsculas), inicializarlo localmente y publicarlo en GitHub/GitLab antes de iniciar el desarrollo.

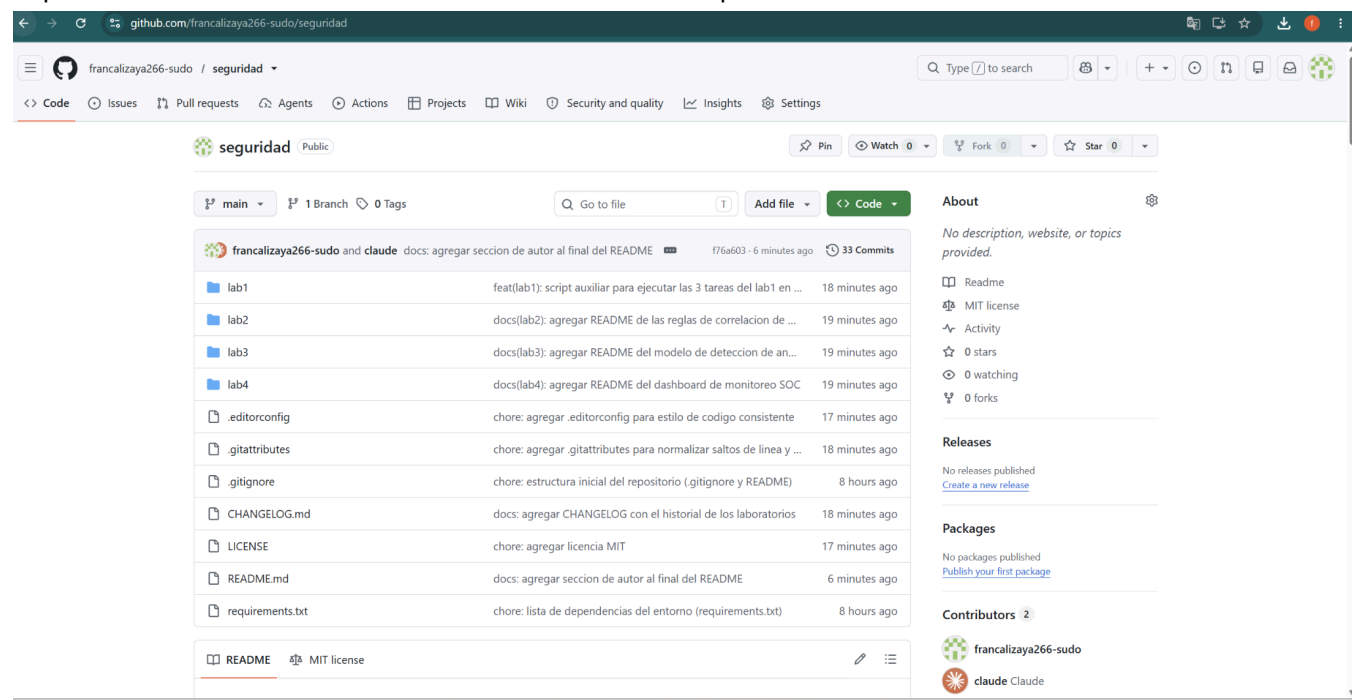
EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
mkdir seguridad && cd seguridad
git init
mkdir -p lab1/evidencias lab1/graficas lab2/evidencias lab3/evidencias lab4/evidencias
touch README.md
git add README.md
git commit -m "Estructura inicial del repositorio - examen final"
git remote add origin https://github.com/francalizaya266-sudo/seguiridad.git
git push -u origin main
```

Objetivo: Crear y publicar el repositorio Git con la estructura de carpetas exigida por el examen antes de iniciar el desarrollo de los laboratorios, cumpliendo el requisito de versionado obligatorio.

Resultado obtenido:

Repositorio creado en GitHub con la estructura inicial de carpetas visible.



El archivo README.md completo (versiones instaladas, comandos de configuración y pasos de reproducción) se entrega junto a esta guía como README.md y debe copiarse a la raíz del repositorio, completando los campos marcados.

2.7 Instalación del entorno Python (Laboratorios 1 y 3)

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
sudo apt update
sudo apt install -y python3 python3-pip python3-venv
```



```
# Entorno virtual dedicado al examen (evita conflictos con paquetes del sistema)
python3 -m venv ~/venv-examen
source ~/venv-examen/bin/activate

pip install --upgrade pip
pip install pandas numpy scikit-learn matplotlib seaborn joblib jupyter

python3 --version
pip freeze | tee ~/seguridad/requirements.txt
```

Objetivo: Instalar Python 3.11+ y las librerías necesarias para los scripts del Laboratorio 1 y el notebook del Laboratorio 3, dejando registrado el listado exacto de versiones en requirements.txt para documentarlo en el README.

Resultado obtenido:

```
pillow==12.2.0
platformdirs==4.10.0
prometheus_client==0.25.0
prompt_toolkit==3.0.52
psutil==7.2.2
ptyprocess==0.7.0
pure_eval==0.2.3
pycparser==3.0
Pygments==2.20.0
pyparsing==3.3.2
python-dateutil==2.9.0.post0
python-json-logger==4.1.0
PyYAML==6.0.3
pyzmq==27.1.0
referencing==0.37.0
requests==2.34.2
rfc3339-validator==0.1.4
rfc3986-validator==0.1.1
rfc3987-syntax==1.1.0
rpds-py==2026.6.3
scikit-learn==1.9.0
scipy==1.18.0
seaborn==0.13.2
Send2Trash==2.1.0
six==1.17.0
soupsieve==2.8.4
stack-data==0.6.3
terminado==0.18.1
threadpoolctl==3.6.0
tinycss2==1.5.1
tornado==6.5.7
traitlets==5.15.1
typing_extensions==4.15.0
tzdata==2026.2
uri-template==1.3.0
urllib3==2.7.0
wcwidth==0.8.2
webcolors==25.10.0
webencodings==0.5.1
websocket-client==1.9.0
widgetsnbextension==4.0.15
```

Recordar activar el entorno virtual (`source ~/venv-examen/bin/activate`) cada vez que se abra una nueva terminal antes de ejecutar los scripts o Jupyter.

2.8 Instalación de Wazuh (Manager + Indexer + Dashboard, modo all-in-one)

Se utiliza el script oficial de instalación asistida de Wazuh, que instala y configura en una sola VM los tres componentes necesarios: Wazuh Manager (motor de reglas y correlación, usado en el Laboratorio 2), Wazuh Indexer (basado en OpenSearch, almacena las alertas) y Wazuh Dashboard (basado en OpenSearch Dashboards, usado como herramienta de visualización en el Laboratorio 4).

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
curl -sO https://packages.wazuh.com/4.9/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
CREDENCIALES:
User: admin
Password: F*ovo?smzclRdN4JuwXyL2GJuTDN.pe+
```

Objetivo: Instalar la pila completa de Wazuh (Manager, Indexer y Dashboard) en modo all-in-one. El proceso toma varios minutos y al finalizar imprime en consola la contraseña generada para el usuario admin del Dashboard: guardarla de inmediato.

Resultado obtenido:

```
30/06/2026 22:12:37 INFO: Wazuh dashboard installation finished.
30/06/2026 22:12:37 INFO: Wazuh dashboard post-install configuration finished.
30/06/2026 22:12:37 INFO: Starting service wazuh-dashboard.
30/06/2026 22:12:39 INFO: wazuh-dashboard service started.
30/06/2026 22:12:41 INFO: Updating the internal users.
30/06/2026 22:12:51 INFO: A backup of the internal users has been saved in the /etc/wazuh-indexer/internalusers-backup folder.

30/06/2026 22:13:11 INFO: The filebeat.yml file has been updated to use the Filebeat Keystore username and password.
30/06/2026 22:14:18 INFO: Initializing Wazuh dashboard web application.
30/06/2026 22:14:19 INFO: Wazuh dashboard web application not yet initialized. Waiting...
30/06/2026 22:14:35 INFO: Wazuh dashboard web application not yet initialized. Waiting...
30/06/2026 22:14:58 INFO: Wazuh dashboard web application initialized.
30/06/2026 22:14:58 INFO: --- Summary ---
30/06/2026 22:14:58 INFO: You can access the web interface https://<wazuh-dashboard-ip>:443
User: admin
Password: F*ovo7smzcLrdN4JuwXyL2GJuTDN.pe+
30/06/2026 22:14:58 INFO: Installation finished.
```

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-indexer
sudo systemctl status wazuh-dashboard
/var/ossec/bin/wazuh-control info
```

Objetivo: Verificar que los tres servicios de Wazuh están activos (active/running) y registrar la versión instalada para el README.

Resultado obtenido:

Salida de los tres systemctl status mostrando active (running), evidencia del entorno funcional.

SCR-
ENV1

```
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-06-30 22:13:59 UTC; 1min 48s ago
     Tasks: 173 (Limit: 9431)
    Memory: 496.0M (peak: 496.0M)
       CPU: 1min 14.012s
    CGroup: /system.slice/wazuh-manager.service
            └─57635 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
            └─57636 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
            └─57639 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
            └─57643 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
            └─57683 /var/ossec/bin/wazuh-authd
            └─57699 /var/ossec/bin/wazuh-db
            └─57709 /var/ossec/bin/wazuh-execd
            └─57720 /var/ossec/bin/wazuh-analysisd
            └─57729 /var/ossec/bin/wazuh-syscheckd
            └─57743 /var/ossec/bin/wazuh-remoted
            └─57767 /var/ossec/bin/wazuh-logcollector
            └─57787 /var/ossec/bin/wazuh-monitord
            └─57884 /var/ossec/bin/wazuh-modulesd

Jun 30 22:13:52 VM1-SIEM env[57573]: Started wazuh-analysisd...
Jun 30 22:13:52 VM1-SIEM env[57573]: Started wazuh-syscheckd...
Jun 30 22:13:53 VM1-SIEM env[57573]: Started wazuh-remoted...
Jun 30 22:13:54 VM1-SIEM env[57573]: Started wazuh-logcollector...
Jun 30 22:13:56 VM1-SIEM env[57573]: Started wazuh-monitord...
Jun 30 22:13:56 VM1-SIEM env[57881]: 2026/06/30 22:13:56 wazuh-modulesd:router: INFO: Loaded router module.
Jun 30 22:13:56 VM1-SIEM env[57881]: 2026/06/30 22:13:56 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
Jun 30 22:13:57 VM1-SIEM env[57573]: Started wazuh-modulesd...
Jun 30 22:13:59 VM1-SIEM env[57573]: Completed.
Jun 30 22:13:59 VM1-SIEM systemd[1]: Started wazuh-manager.service - Wazuh manager.
```

```
● wazuh-dashboard.service - wazuh-dashboard
   Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-06-30 22:14:03 UTC; 1min 55s ago
     Main PID: 58513 (node)
       Tasks: 11 (limit: 9431)
    Memory: 167.6M (peak: 275.2M)
       CPU: 18.292s
   CGroup: /system.slice/wazuh-dashboard.service
           └─58513 /usr/share/wazuh-dashboard/node/bin/node /usr/share/wazuh-dashboard/src/cli/dist

Jun 30 22:14:18 VM1-SIEM opensearch-dashboards[58513]: {"type":"log","@timestamp":"2026-06-30T22:14:18Z","tags":["info","plugins","wazuh","monitoring"],"pid":58513,"message":
Jun 30 22:14:19 VM1-SIEM opensearch-dashboards[58513]: {"type":"log","@timestamp":"2026-06-30T22:14:19Z","tags":["listening","info"],"pid":58513,"message":"Server running at
Jun 30 22:14:19 VM1-SIEM opensearch-dashboards[58513]: {"type":"log","@timestamp":"2026-06-30T22:14:19Z","tags":["info","http","server","OpenSearchDashboards"],"pid":58513,"
Jun 30 22:14:19 VM1-SIEM opensearch-dashboards[58513]: {"type":"log","@timestamp":"2026-06-30T22:14:19Z","tags":["info","plugins","wazuh","cron-scheduler"],"pid":58513,"mess
Jun 30 22:14:19 VM1-SIEM opensearch-dashboards[58513]: {"type":"log","@timestamp":"2026-06-30T22:14:19Z","tags":["info","plugins","wazuh","monitoring"],"pid":58513,"message":
Jun 30 22:14:34 VM1-SIEM opensearch-dashboards[58513]: [agentkeepalive:deprecated] options.freeSocketKeepAliveTimeout is deprecated, please use options.freeSocketTimeout ins
Jun 30 22:14:35 VM1-SIEM opensearch-dashboards[58513]: {"type":"response","@timestamp":"2026-06-30T22:14:34Z","tags":[],"pid":58513,"method":"get","statusCode":200,"req":{"u
Jun 30 22:15:00 VM1-SIEM opensearch-dashboards[58513]: {"type":"log","@timestamp":"2026-06-30T22:15:00Z","tags":["info","plugins","wazuh","monitoring"],"pid":58513,"message":
Jun 30 22:15:01 VM1-SIEM opensearch-dashboards[58513]: {"type":"log","@timestamp":"2026-06-30T22:15:01Z","tags":["error","opensearch","data"],"pid":58513,"message":["resourc
Jun 30 22:15:01 VM1-SIEM opensearch-dashboards[58513]: {"type":"log","@timestamp":"2026-06-30T22:15:01Z","tags":["info","plugins","wazuh","cron-scheduler"],"pid":58513,"mess
```

El Wazuh Dashboard queda accesible en <https://127.0.0.1:8443> desde el navegador del equipo anfitrión (aceptar el certificado autofirmado). Con esto el entorno completo está listo: Wazuh para los Laboratorios 2 y 4, y Python/Jupyter para los Laboratorios 1 y 3.

Laboratorio 1 — Análisis Forense de Logs con Python (5 pts)

En este laboratorio se trabaja exclusivamente con Python, sin depender de Wazuh ni de las VMs de ataque. Se parte de dos archivos de log ya provistos por el repositorio del curso (auth.log y access.log) y se construyen tres scripts que automatizan la detección manual que haría un analista SOC. El resultado de este laboratorio es comparable entre estudiantes porque el dataset de entrada es idéntico.

Tarea 1.1 — Parseo y estadísticas de auth.log (1.5 pts)

Se copian los datasets oficiales del curso a la carpeta lab1/ del repositorio propio.

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
git clone --depth 1 https://github.com/abelthf/examen_final_seguridad_informatica.git
/tmp/curso
cp /tmp/curso/lab1/auth.log lab1/auth.log
cp /tmp/curso/lab1/access.log lab1/access.log
cp /tmp/curso/lab2/simular_bruteforce.sh lab2/simular_bruteforce.sh
cp /tmp/curso/lab3/network_traffic.csv lab3/network_traffic.csv
wc -l lab1/auth.log lab1/access.log
```

Objetivo: Obtener los datasets oficiales del curso para garantizar que los resultados de análisis sean comparables con los de los demás estudiantes.

Resultado obtenido:

Código completo de analizar_ssh.py (entregado también como archivo suelto):

```
#!/usr/bin/env python3
"""
analizar_ssh.py
-----
Examen Final - Seguridad Informatica - Unidad IV
Laboratorio 1 - Tarea 1.1: Parseo y estadísticas de auth.log
Autor: Fran Franklin Calizaya Apaza

Lee lab1/auth.log, identifica intentos de autenticacion SSH fallidos
("Failed password"), cuenta los intentos por IP de origen, genera un
ranking Top 10, emite alertas en consola para IPs con mas de 50 intentos
y exporta el resultado a reporte_ssh.json.
"""

import re
import json
from collections import Counter
from datetime import datetime
from pathlib import Path

LOG_PATH = Path("auth.log")
REPORT_PATH = Path("reporte_ssh.json")
UMBRAL_ALERTA = 50

# Coincide con lineas tipo:
# Mar 15 02:14:23 server sshd[12345]: Failed password for invalid user admin from 203.0.113.45
# port 51234 ssh2
```

```

# Mar 15 02:14:23 server sshd[12345]: Failed password for root from 203.0.113.45 port 51234
ssh2
PATRON_FALLO = re.compile(
    r"Failed password for (?:(invalid user )?\S+ from "
    r"(?P<ip>\d{1,3}(\.?\d{1,3}){3})"
)

def leer_log(path: Path) -> list[str]:
    if not path.exists():
        raise FileNotFoundError(
            f"No se encontro {path}. Copie auth.log a la carpeta lab1/ antes de ejecutar."
        )
    with path.open("r", encoding="utf-8", errors="ignore") as f:
        return f.readlines()

def contar_intentos_fallidos(lineas: list[str]) -> Counter:
    contador = Counter()
    for linea in lineas:
        match = PATRON_FALLO.search(linea)
        if match:
            contador[match.group("ip")] += 1
    return contador

def generar_alertas(contador: Counter) -> list[dict]:
    ips_sospechosas = []
    for ip, intentos in contador.most_common():
        alerta = intentos > UMBRAL_ALERTA
        if alerta:
            print(f"[ALERTA] IP: {ip} - {intentos} intentos fallidos - Posible ataque de fuerza
bruta")
            ips_sospechosas.append({"ip": ip, "intentos": intentos, "alerta": alerta})
    return ips_sospechosas

def main():
    lineas = leer_log(LOG_PATH)
    contador = contar_intentos_fallidos(lineas)
    total_fallidos = sum(contador.values())

    print(f"Total de lineas analizadas: {len(lineas)}")
    print(f"Total de intentos fallidos detectados: {total_fallidos}")
    print(f"IPs distintas con intentos fallidos: {len(contador)}\n")

    print("=== Top 10 IPs con mas intentos fallidos ===")
    for posicion, (ip, intentos) in enumerate(contador.most_common(10), start=1):
        print(f"{posicion:>2}. {ip:<15} -> {intentos} intentos")
    print()

    print("=== Verificacion de umbral de fuerza bruta (>50 intentos) ===")
    ips_sospechosas = generar_alertas(contador)

    reporte = {
        "fecha_analisis": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "total_intentos_fallidos": total_fallidos,
        "ips_sospechosas": sorted(
            ips_sospechosas, key=lambda x: x["intentos"], reverse=True
        )[:10],
    }

    with REPORT_PATH.open("w", encoding="utf-8") as f:

```

```
        json.dump(reporte, f, indent=2, ensure_ascii=False)

    print(f"\nReporte exportado a {REPORT_PATH.resolve()}")

if __name__ == "__main__":
    main()
```

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
source ~/venv-examen/bin/activate
cd lab1
python3 analizar_ssh.py
```

Objetivo: Ejecutar el script de análisis de autenticación SSH y verificar que detecta correctamente las IPs sospechosas y genera el reporte JSON con la estructura exigida.

Resultado obtenido:

```
=== Top 10 IPs con mas intentos fallidos ===
1. 45.33.32.156 -> 120 intentos
2. 193.32.162.55 -> 58 intentos
3. 91.240.118.172 -> 30 intentos
4. 172.16.0.70 -> 4 intentos
5. 192.168.1.90 -> 4 intentos
6. 172.16.0.227 -> 3 intentos
7. 172.16.0.76 -> 3 intentos
8. 172.16.0.182 -> 2 intentos
9. 10.0.222.18 -> 2 intentos
10. 192.168.1.108 -> 2 intentos
```

Terminal ejecutando python analizar_ssh.py con las líneas [ALERTA] visibles en consola.


SCR-1.1a

```
Total de líneas analizadas: 500
Total de intentos fallidos detectados: 253
IPs distintas con intentos fallidos: 35

=== Top 10 IPs con mas intentos fallidos ===
1. 45.33.32.156 -> 120 intentos
2. 193.32.162.55 -> 58 intentos
3. 91.240.118.172 -> 30 intentos
4. 172.16.0.70 -> 4 intentos
5. 192.168.1.90 -> 4 intentos
6. 172.16.0.227 -> 3 intentos
7. 172.16.0.76 -> 3 intentos
8. 172.16.0.182 -> 2 intentos
9. 10.0.222.18 -> 2 intentos
10. 192.168.1.108 -> 2 intentos

=== Verificación de umbral de fuerza bruta (>50 intentos) ===
[ALERTA] IP: 45.33.32.156 - 120 intentos fallidos - Posible ataque de fuerza bruta
[ALERTA] IP: 193.32.162.55 - 58 intentos fallidos - Posible ataque de fuerza bruta
```

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
cat reporte_ssh.json
```

Resultado obtenido:
SCR-1.1b

Contenido del archivo reporte_ssh.json (terminal con cat o editor).

```
(venv-examen) alexserver@VM1-SIEM:~/examen-practico-coila/lab1$ cat reporte_ssh.json
{
  "fecha_analisis": "2026-06-30 22:38:42",
  "total_intentos_fallidos": 253,
  "ips_sospechosas": [
    {
      "ip": "45.33.32.156",
      "intentos": 120,
      "alerta": true
    },
    {
      "ip": "193.32.162.55",
      "intentos": 58,
      "alerta": true
    },
    {
      "ip": "91.240.118.172",
      "intentos": 30,
      "alerta": false
    },
    {
      "ip": "172.16.0.70",
      "intentos": 4,
      "alerta": false
    },
    {
      "ip": "192.168.1.90",
      "intentos": 4,
      "alerta": false
    },
    {
      "ip": "172.16.0.227",
      "intentos": 3,
      "alerta": false
    },
    {
      "ip": "172.16.0.76",
      "intentos": 3,
      "alerta": false
    },
    {
      "ip": "172.16.0.182",

```

Tarea 1.2 — Análisis de access.log (1.5 pts)

Código completo de analizar_web.py:

```
#!/usr/bin/env python3
"""
analizar_web.py
-----
Examen Final - Seguridad Informatica - Unidad IV
Laboratorio 1 - Tarea 1.2: Analisis de access.log
Autor: Fran Franklin Calizaya Apaza

Parsea lab1/access.log en formato Combined Log Format de Apache,
detecta patrones de escaneo de directorios (> 20 rutas distintas en
< 60 segundos desde la misma IP), agrupa codigos de respuesta 4xx/5xx
por IP, detecta posibles intentos de SQL Injection en la URL, y
exporta el resultado a reporte_web.json.
"""

import re
import json
from collections import defaultdict
from datetime import datetime
from pathlib import Path
from urllib.parse import unquote

LOG_PATH = Path("access.log")
REPORT_PATH = Path("reporte_web.json")

VENTANA_ESCANEEO_SEG = 60
UMBRAL_RUTAS_ESCANEEO = 20

PATRONES_SQLI = ["UNION", "SELECT", "--", "OR 1=1", "'"]

# Coincide con el Combined Log Format estandar de Apache. La URL se captura
```

```

# de forma no-codiciosa hasta el literal " HTTP/x.x" (en vez de \S+) porque
# algunos payloads de prueba (ej. SQL Injection) llegan con espacios sin
# codificar dentro de la URL (ej. "GET /api?id=1 OR 1=1 HTTP/1.1"), y un
# patron basado en \S+ los descartaria por completo.
PATRON_CLF = re.compile(
    r'^(?P<ip>\S+) \S+ \S+ \[(?P<fecha>[^\]]+)\]'
    r'"(?P<metodo>\S+) (?P<url>.+?) HTTP/\d\.\d"'
    r'(?P<status>\d{3}) (?P<size>\S+)'
)

FORMATO_FECHA_APACHE = "%d/%b/%Y:%H:%M:%S %z"

def parsear_log(path: Path) -> list[dict]:
    if not path.exists():
        raise FileNotFoundError(
            f"No se encontro {path}. Copie access.log a la carpeta lab1/ antes de ejecutar."
        )
    registros = []
    with path.open("r", encoding="utf-8", errors="ignore") as f:
        for linea in f:
            match = PATRON_CLF.search(linea)
            if not match:
                continue
            datos = match.groupdict()
            try:
                datos["timestamp"] = datetime.strptime(
                    datos["fecha"], FORMATO_FECHA_APACHE
                )
            except ValueError:
                continue
            registros.append(datos)
    return registros

def detectar_escaneo_directorios(registros: list[dict]) -> list[dict]:
    """Mas de 20 peticiones a rutas distintas en menos de 60s desde la misma IP."""
    por_ip = defaultdict(list)
    for r in registros:
        por_ip[r["ip"]].append(r)

    hallazgos = []
    for ip, eventos in por_ip.items():
        eventos.sort(key=lambda x: x["timestamp"])
        inicio = 0
        for fin in range(len(eventos)):
            while (eventos[fin]["timestamp"] - eventos[inicio]["timestamp"]).total_seconds() >
VENTANA_ESCANEO_SEG:
                inicio += 1
            ventana = eventos[inicio:fin + 1]
            rutas_distintas = {e["url"] for e in ventana}
            if len(rutas_distintas) > UMBRAL_RUTAS_ESCANEO:
                hallazgos.append({
                    "ip": ip,
                    "rutas_distintas": len(rutas_distintas),
                    "inicio": ventana[0]["timestamp"].isoformat(),
                    "fin": ventana[-1]["timestamp"].isoformat(),
                })
            break # con un hallazgo por IP es suficiente para el reporte
    return hallazgos

def agrupar_codigos_error(registros: list[dict]) -> dict:

```



```

por_ip = defaultdict(lambda: defaultdict(int))
for r in registros:
    codigo = int(r["status"])
    if codigo >= 400:
        por_ip[r["ip"]][str(codigo)] += 1
return {ip: dict(codigos) for ip, codigos in por_ip.items()}

def detectar_sqli(registros: list[dict]) -> list[dict]:
    hallazgos = []
    for r in registros:
        url_decodificada = unquote(r["url"]).upper()
        patrones_encontrados = [p for p in PATRONES_SQLI if p.upper() in url_decodificada]
        if patrones_encontrados:
            hallazgos.append({
                "ip": r["ip"],
                "url": r["url"],
                "patrones": patrones_encontrados,
                "timestamp": r["timestamp"].isoformat(),
            })
    return hallazgos

def main():
    registros = parsear_log(LOG_PATH)
    print(f"Total de líneas parseadas correctamente: {len(registros)}")

    print("\n=== Deteccion de escaneo de directorios ===")
    escaneos = detectar_escaneo_directorios(registros)
    if escaneos:
        for h in escaneos:
            print(f"[ALERTA] Escaneo detectado desde {h['ip']}: {h['rutas_distintas']} rutas distintas "
                  f"entre {h['inicio']} y {h['fin']}")
    else:
        print("No se detectaron patrones de escaneo de directorios.")

    print("\n=== Codigos de respuesta 4xx/5xx agrupados por IP ===")
    errores_por_ip = agrupar_codigos_error(registros)
    for ip, codigos in sorted(errores_por_ip.items(), key=lambda x: sum(x[1].values()),
                             reverse=True)[:10]:
        print(f"{ip}: {codigos}")

    print("\n=== Deteccion de posibles intentos de SQL Injection ===")
    sqli = detectar_sqli(registros)
    if sqli:
        for h in sqli[:20]:
            print(f"[ALERTA] Posible SQLi desde {h['ip']}: {h['url']} -> patrones: {h['patrones']}")
            if len(sqli) > 20:
                print(f"... y {len(sqli) - 20} coincidencias adicionales (ver reporte_web.json)")
    else:
        print("No se detectaron patrones de SQL Injection.")

    reporte = {
        "fecha_analisis": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "total_peticones_analizadas": len(registros),
        "escaneo_directorios": escaneos,
        "codigos_error_por_ip": errores_por_ip,
        "intentos_sql_injection": sqli,
    }

    with REPORT_PATH.open("w", encoding="utf-8") as f:

```

```
json.dump(reporte, f, indent=2, ensure_ascii=False)

print(f"\nReporte exportado a {REPORT_PATH.resolve()}")

if __name__ == "__main__":
    main()
```

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
python3 analizar_web.py
```

Objetivo: Verificar que el script detecta correctamente patrones de escaneo de directorios y de SQL Injection, y agrupa adecuadamente los códigos de error por IP.

Resultado obtenido:

Terminal ejecutando python analizar_web.py mostrando detecciones de escaneo y SQLi.

SCR-
1.2a

```
Total de líneas parseadas correctamente: 1000

=== Deteccion de escaneo de directorios ===
No se detectaron patrones de escaneo de directorios.

=== Codigos de respuesta 4xx/5xx agrupados por IP ===
45.32.32.156: {'404': 34, '403': 10, '500': 3}
193.32.162.55: {'500': 14, '400': 4}
89.210.135.99: {'404': 10, '500': 8}
122.153.40.102: {'500': 10, '404': 7}
115.88.85.249: {'500': 8, '404': 5}
185.220.181.45: {'500': 7, '404': 6}
147.242.19.45: {'500': 5, '404': 7}
91.240.118.172: {'404': 6, '500': 6}
183.17.75.31: {'500': 8, '404': 4}
211.75.132.82: {'404': 6, '500': 6}

=== Deteccion de posibles intentos de SQL Injection ===
[ALERTA] Posible SQLi desde 193.32.162.55: /login?user=admin'--&pass=x -> patrones: ['--', '']
[ALERTA] Posible SQLi desde 193.32.162.55: /api/data?filter=1 OR 1=1 -> patrones: ['OR 1=1']
[ALERTA] Posible SQLi desde 193.32.162.55: /login?user=admin'--&pass=x -> patrones: ['--', '']
[ALERTA] Posible SQLi desde 193.32.162.55: /products?cat=1; SELECT * FROM users-- -> patrones: ['SELECT', '--']
[ALERTA] Posible SQLi desde 193.32.162.55: /search?q=' OR 1=1-- -> patrones: ['--', 'OR 1=1', '']
[ALERTA] Posible SQLi desde 193.32.162.55: /api/v1/users?id=1 UNION SELECT username,password FROM users-- -> patrones: ['UNION', 'SELECT', '--']
[ALERTA] Posible SQLi desde 193.32.162.55: /search?q=' OR 1=1-- -> patrones: ['--', 'OR 1=1', '']
[ALERTA] Posible SQLi desde 193.32.162.55: /item?id=1 UNION SELECT null,table_name FROM information_schema.tables-- -> patrones: ['UNION', 'SELECT', '--']
[ALERTA] Posible SQLi desde 193.32.162.55: /products?cat=1; SELECT * FROM users-- -> patrones: ['SELECT', '--']
[ALERTA] Posible SQLi desde 193.32.162.55: /api/v1/users?id=1 UNION SELECT username,password FROM users-- -> patrones: ['UNION', 'SELECT', '--']
[ALERTA] Posible SQLi desde 193.32.162.55: /products?cat=1; SELECT * FROM users-- -> patrones: ['SELECT', '--']
[ALERTA] Posible SQLi desde 193.32.162.55: /search?q=' OR 1=1-- -> patrones: ['--', 'OR 1=1', '']
[ALERTA] Posible SQLi desde 193.32.162.55: /api/data?filter=1 OR 1=1 -> patrones: ['OR 1=1']
[ALERTA] Posible SQLi desde 193.32.162.55: /products?cat=1; SELECT * FROM users-- -> patrones: ['SELECT', '--']
[ALERTA] Posible SQLi desde 193.32.162.55: /login?user=admin'--&pass=x -> patrones: ['--', '']
[ALERTA] Posible SQLi desde 193.32.162.55: /login?user=admin'--&pass=x -> patrones: ['--', '']
[ALERTA] Posible SQLi desde 193.32.162.55: /api/data?filter=1 OR 1=1 -> patrones: ['OR 1=1']
[ALERTA] Posible SQLi desde 193.32.162.55: /api/data?filter=1 OR 1=1 -> patrones: ['OR 1=1']
[ALERTA] Posible SQLi desde 193.32.162.55: /item?id=1 UNION SELECT null,table_name FROM information_schema.tables-- -> patrones: ['UNION', 'SELECT', '--']
[ALERTA] Posible SQLi desde 193.32.162.55: /item?id=1 UNION SELECT null,table_name FROM information_schema.tables-- -> patrones: ['UNION', 'SELECT', '--']
... y 4 coincidencias adicionales (ver reporte_web.json)
```

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
cat reporte_web.json
```

Resultado obtenido:

SCR-
1.2b

Contenido del archivo reporte_web.json.

```

{
  "ip": "193.32.162.55",
  "url": "/search?q=' OR 1=1--",
  "patrones": [
    "--",
    "OR 1=1",
    ""
  ],
  "timestamp": "2024-06-14T05:32:32+00:00"
},
{
  "ip": "193.32.162.55",
  "url": "/item?id=1 UNION SELECT null,table_name FROM information_schema.tables--",
  "patrones": [
    "UNION",
    "SELECT",
    "--"
  ],
  "timestamp": "2024-06-14T05:33:04+00:00"
},
{
  "ip": "193.32.162.55",
  "url": "/api/v1/users?id=1 UNION SELECT username,password FROM users--",
  "patrones": [
    "UNION",
    "SELECT",
    "--"
  ],
  "timestamp": "2024-06-14T05:31:36+00:00"
},
{
  "ip": "193.32.162.55",
  "url": "/api/v1/users?id=1 UNION SELECT username,password FROM users--",
  "patrones": [
    "UNION",
    "SELECT",
    "--"
  ],
  "timestamp": "2024-06-14T05:30:00+00:00"
}
]

```

Tarea 1.3 — Visualización (2 pts)

Código completo de visualizar.py:

```

#!/usr/bin/env python3
"""
visualizar.py
-----
Examen Final - Seguridad Informatica - Unidad IV
Laboratorio 1 - Tarea 1.3: Visualizacion
Autor: Fran Franklin Calizaya Apaza

Genera tres graficas a partir de reporte_ssh.json y access.log:
1. Barras: Top 10 IPs con mas intentos fallidos SSH.
2. Línea de tiempo: peticiones HTTP por hora.
3. Mapa de calor: peticiones HTTP por hora y codigo de respuesta.

Requiere: pip install matplotlib seaborn pandas --break-system-packages
"""

import json
import re
from collections import defaultdict
from datetime import datetime
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

REPORTE_SSH = Path("reporte_ssh.json")
ACCESS_LOG = Path("access.log")
SALIDA = Path("graficas")
SALIDA.mkdir(exist_ok=True)

```

```

PATRON_CLF = re.compile(
    r'^(?P<ip>\S+) \S+ \S+ \[(?P<fecha>[^\]]+\)\]'
    r'""(?P<metodo>\S+) (?P<url>.+?) HTTP/\d\.\d"'
    r'(?P<status>\d{3}) (?P<size>\S+)'
)
FORMATO_FECHA_APACHE = "%d/%b/%Y:%H:%M:%S %z"

def cargar_top10_ssh():
    with REPORTE_SSH.open(encoding="utf-8") as f:
        data = json.load(f)
        ips = [item["ip"] for item in data["ips_sospechosas"][:10]]
        intentos = [item["intentos"] for item in data["ips_sospechosas"][:10]]
        return ips, intentos

def cargar_eventos_web():
    eventos = []
    with ACCESS_LOG.open(encoding="utf-8", errors="ignore") as f:
        for linea in f:
            m = PATRON_CLF.search(linea)
            if not m:
                continue
            try:
                ts = datetime.strptime(m.group("fecha"), FORMATO_FECHA_APACHE)
            except ValueError:
                continue
            eventos.append({"timestamp": ts, "status": int(m.group("status"))})
    return pd.DataFrame(eventos)

def grafico_top10_ssh(ips, intentos):
    plt.figure(figsize=(10, 6))
    sns.barplot(x=intentos, y=ips, hue=ips, palette="Reds_r", legend=False)
    plt.title("Top 10 IPs con mas intentos fallidos SSH", fontsize=14, fontweight="bold")
    plt.xlabel("Intentos fallidos")
    plt.ylabel("Direccion IP")
    plt.tight_layout()
    plt.savefig(SALIDA / "top10_ssh.png", dpi=150)
    plt.close()
    print(f"Generado: {SALIDA / 'top10_ssh.png'}")

def grafico_timeline_http(df: pd.DataFrame):
    df["hora"] = df["timestamp"].dt.floor("h")
    serie = df.groupby("hora").size()
    plt.figure(figsize=(12, 5))
    serie.plot(kind="line", marker="o", color="#2E75B6")
    plt.title("Peticiones HTTP por hora", fontsize=14, fontweight="bold")
    plt.xlabel("Hora")
    plt.ylabel("Numero de peticiones")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig(SALIDA / "timeline_http.png", dpi=150)
    plt.close()
    print(f"Generado: {SALIDA / 'timeline_http.png'}")

def grafico_heatmap_http(df: pd.DataFrame):
    df["hora_del_dia"] = df["timestamp"].dt.hour
    codigos_interes = [200, 301, 404, 500]
    df_filtrado = df[df["status"].isin(codigos_interes)]
    tabla = pd.crosstab(df_filtrado["hora_del_dia"], df_filtrado["status"])

```

```
tabla = tabla.reindex(columns=codigos_interes, fill_value=0)

plt.figure(figsize=(8, 8))
sns.heatmap(tabla, annot=True, fmt="d", cmap="YlOrRd", linewidths=0.5)
plt.title("Peticiones HTTP por hora y codigo de respuesta", fontsize=14, fontweight="bold")
plt.xlabel("Codigo de respuesta")
plt.ylabel("Hora del dia")
plt.tight_layout()
plt.savefig(SALIDA / "heatmap_http.png", dpi=150)
plt.close()
print(f"Generado: {SALIDA / 'heatmap_http.png'}")

def main():
    ips, intentos = cargar_top10_ssh()
    grafico_top10_ssh(ips, intentos)

    df = cargar_eventos_web()
    grafico_timeline_http(df)
    grafico_heatmap_http(df)

    print("\nLas 3 graficas fueron guardadas en la carpeta graficas/")

if __name__ == "__main__":
    main()
```

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
pip install matplotlib seaborn pandas
python3 visualizar.py
ls graficas/
```

Objetivo: Generar evidencia visual que facilite la interpretación de los hallazgos numéricos obtenidos en las tareas 1.1 y 1.2: top10_ssh.png, timeline_http.png y heatmap_http.png.

Resultado obtenido:

Las 3 gráficas generadas: top10_ssh.png, timeline_http.png, heatmap_http.png.

Laboratorio 2 — Reglas de Correlación en Wazuh (4 pts)

Este laboratorio traslada la lógica de detección del Laboratorio 1 a un motor SIEM real. En lugar de calcular manualmente con Python, se configuran reglas XML en Wazuh para que el propio sistema dispare alertas automáticas en tiempo real ante un ataque de fuerza bruta SSH y ante un escenario de exfiltración de datos. La validación se realiza ejecutando el script oficial `simular_bruteforce.sh` en la propia VM Wazuh-SIEM (inyección de eventos vía syslog).

Tarea 2.1 — Regla: Brute Force SSH (1.5 pts)

Lógica: la regla base de Wazuh 5716 ("sshd: authentication failed") ya detecta cada fallo individual. La regla 100001 cuenta cuántas veces se dispara 5716 desde la misma IP (same_source_ip) dentro de 60 segundos; si ocurren 10 o más eventos (frequency="10" timeframe="60"), se eleva la severidad a nivel 10.

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
sudo nano /var/ossec/etc/rules/local_rules_ssh.xml
```

Contenido completo de local_rules_ssh.xml (también entregado como archivo suelto):

```
<!--
  local_rules_ssh.xml
  Examen Final - Seguridad Informatica - Unidad IV
  Laboratorio 2 - Tarea 2.1: Regla de fuerza bruta SSH
  Autor: Fran Franklin Calizaya Apaza

  Logica: la regla base de Wazuh 5716 ("sshd: authentication failed")
  ya detecta cada fallo individual de autenticacion SSH. Esta regla de
  correlacion cuenta cuantas veces se dispara 5716 desde la MISMA IP de
  origen (same_source_ip) dentro de una ventana de 60 segundos. Si
  ocurren 10 o mas eventos en ese intervalo (frequency="10"
  timeframe="60"), se considera fuerza bruta y se eleva la severidad a
  nivel 10.
-->
<group name="local,ssh,">

  <rule id="100001" level="10" frequency="10" timeframe="60">
    <if_matched_sid>5716</if_matched_sid>
    <same_source_ip />
    <description>Ataque de fuerza bruta SSH detectado desde $(srcip)</description>
    <group>authentication_failures,brute_force,</group>
  </rule>

</group>
```

Objetivo: Configurar la regla de correlación que permita a Wazuh detectar automáticamente patrones de fuerza bruta SSH con severidad alta (nivel 10).

Resultado obtenido:

Tarea 2.2 — Regla: Exfiltración de datos (1.5 pts)

Lógica de correlación en dos etapas (detallada como comentario dentro del propio XML): un login SSH exitoso fuera de horario laboral (22:00–06:00) seguido, dentro de la misma hora y desde la misma IP, de una transferencia saliente superior a 500 MB, dispara la alerta crítica (nivel 14).

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
sudo nano /var/ossec/etc/rules/local_rules_exfil.xml
```

Contenido completo de local_rules_exfil.xml (también entregado como archivo suelto):

```
<!--
local_rules_exfil.xml
Examen Final - Seguridad Informatica - Unidad IV
Laboratorio 2 - Tarea 2.2: Regla compuesta de exfiltracion de datos
Autor: Fran Franklin Calizaya Apaza

Logica de la regla (correlacion en dos etapas):

1) Regla 100012 (nivel bajo, "oculta" en la practica porque solo sirve
   como bloque de construccion): se dispara cuando un login SSH exitoso
   (sid base 5715 del ruleset de Wazuh, "sshd: authentication success")
   ocurre dentro del rango horario 22:00-06:00, es decir, fuera del
   horario laboral. Queda registrada en el grupo "off_hours_access".

2) Regla 100013 (nivel bajo): se dispara cuando el log de trafico
   saliente reporta un campo bytes_sent cuyo valor decimal es mayor o
   igual a 500.000.000 (500 MB). El patron pcre2 acepta numeros de 9
   cifras que comienzan en 5-9 (500000000-999999999) o de 10+ cifras.
   IMPORTANTE: <if_sid>530</if_sid> es un marcador de ejemplo; debe
   reemplazarse por el sid real del decoder/regla que procesa los logs
   de trafico saliente del firewall/proxy en el entorno del estudiante.

3) Regla 100014 (nivel 14, CRITICA): es la regla de correlacion final.
   Se dispara cuando, dentro de la MISMA hora (timeframe="3600"), desde
   la MISMA IP de origen (same_source_ip), ocurre un evento que coincide
   con la regla 100013 (transferencia >500MB, evaluada con <if_sid> sobre
   el evento actual) Y ademas existe un evento previo que coincidio con
   la regla 100012 (login exitoso fuera de horario, evaluado con
   <if_matched_sid> sobre el historial de correlacion). Esta combinacion
   modela el escenario real de exfiltracion: alguien entra fuera de
   horario laboral y, poco despues, saca una gran cantidad de datos.
-->
<group name="local,data_loss_prevention,">

  <!-- Etapa 1: login SSH exitoso fuera de horario laboral -->
  <rule id="100012" level="3" frequency="1" timeframe="86400">
    <if_sid>5715</if_sid>
    <time>22:00 - 06:00</time>
    <description>Login SSH exitoso fuera de horario laboral desde $(srcip)</description>
    <group>off_hours_access,</group>
  </rule>

  <!-- Etapa 2: transferencia de datos saliente superior a 500 MB -->
  <rule id="100013" level="3">
    <if_sid>530</if_sid> <!-- reemplazar por el sid real del origen de logs de trafico saliente
-->
    <field name="bytes_sent" type="pcre2">^([5-9]\d{8}|\d{10,})$</field>
    <description>Transferencia de datos saliente superior a 500 MB detectada desde
$(srcip)</description>
    <group>data_transfer,large_upload,</group>
  </rule>

  <!-- Etapa 3: correlacion critica = etapa 2 actual + etapa 1 previa, misma IP -->
  <rule id="100014" level="14" frequency="1" timeframe="3600">
    <if_sid>100013</if_sid>
    <if_matched_sid>100012</if_matched_sid>
    <same_source_ip />
```

```

    <description>Posible exfiltracion de datos: transferencia >500MB desde $(srcip) tras un login
    exitoso fuera de horario laboral</description>
    <group>data_leak,exfiltration,</group>
  </rule>
</group>

```

Nota: el <id>530</id> de la regla 100013 es un marcador de ejemplo que apunta al sid base que procesa los logs de tráfico saliente (firewall/proxy). Debe reemplazarse por el sid real del decoder/origen configurado en tu entorno; el comentario dentro del XML explica el porqué de cada etapa de la correlación.

Objetivo: Configurar una regla avanzada de correlación compuesta que represente un escenario real de exfiltración de datos tras un acceso fuera de horario.

Resultado obtenido:

```

reemplazarse por el sid real del decoder/regla que procesa los logs
de trafico saliente del firewall/proxy en el entorno del estudiante.

3) Regla 100014 (nivel 14, CRITICA): es la regla de correlacion final.
Se dispara cuando, dentro de la MISMA hora (timeframe="3600"), desde
la MISMA IP de origen (same_source_ip), ocurre un evento que coincide
con la regla 100013 (transferencia >500MB, evaluada con <id> sobre
el evento actual) Y ademas existe un evento previo que coincidio con
la regla 100012 (login exitoso fuera de horario, evaluado con
<id_matched_sid> sobre el historial de correlacion). Esta combinacion
modela el escenario real de exfiltracion: alguien entra fuera de
horario laboral y, poco despues, saca una gran cantidad de datos.
-->
<group name="local_data_loss_prevention,">

  <!-- Etapa 1: login SSH exitoso fuera de horario laboral -->
  <rule id="100012" level="3" frequency="1" timeframe="86400">
    <id>5715</id>
    <time>22:00 - 06:00</time>
    <description>Login SSH exitoso fuera de horario laboral desde $(srcip)</description>
    <group>off_hours_access,</group>
  </rule>

  <!-- Etapa 2: transferencia de datos saliente superior a 500 MB -->
  <rule id="100013" level="3">
    <id>530</id> <!-- reemplazar por el sid real del origen de logs de trafico saliente -->
    <field name="bytes_sent" type="pcre2">^([5-9]\d{8})\d{10,}</field>
    <description>Transferencia de datos saliente superior a 500 MB detectada desde $(srcip)</description>
    <group>data_transfer,large_upload,</group>
  </rule>

  <!-- Etapa 3: correlacion critica = etapa 2 actual + etapa 1 previa, misma IP -->
  <rule id="100014" level="14" frequency="1" timeframe="3600">
    <id>100013</id>
    <id_matched_sid>100012</id_matched_sid>
    <same_source_ip />
    <description>Posible exfiltracion de datos: transferencia >500MB desde $(srcip) tras un login exitoso fuera de horario laboral</description>
    <group>data_leak,exfiltration,</group>
  </rule>
</group>

```

Tarea 2.3 — Prueba y evidencia (1 pt)

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```

sudo xmllint --noout /var/ossec/etc/rules/local_rules_ssh.xml && echo OK
sudo xmllint --noout /var/ossec/etc/rules/local_rules_exfil.xml && echo OK
sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager

```

Objetivo: Validar la sintaxis XML de ambas reglas y reiniciar el Wazuh Manager para que las cargue.

Resultado obtenido:



SCR-
2.1

Salida de systemctl status wazuh-manager con estado active (running).


```

# wazuh-manager.service - Wazuh manager
Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
Active: active (running) since Tue 2026-06-30 23:06:28 UTC; 15s ago
Process: 60947 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
Tasks: 171 (limit: 9431)
Memory: 290.3M (peak: 292.3M)
CPU: 38.432s
CGroup: /system.slice/wazuh-manager.service
├─61009 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─61010 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─61013 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─61016 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─61057 /var/ossec/bin/wazuh-authd
├─61070 /var/ossec/bin/wazuh-db
├─61080 /var/ossec/bin/wazuh-execd
├─61106 /var/ossec/bin/wazuh-analysisd
├─61115 /var/ossec/bin/wazuh-syscheckd
├─61129 /var/ossec/bin/wazuh-remoted
├─61165 /var/ossec/bin/wazuh-logcollector
├─61231 /var/ossec/bin/wazuh-monitord
└─61240 /var/ossec/bin/wazuh-modulesd

Jun 30 23:06:21 VM1-SIEM env[60947]: Started wazuh-analysisd...
Jun 30 23:06:21 VM1-SIEM env[60947]: Started wazuh-syscheckd...
Jun 30 23:06:23 VM1-SIEM env[60947]: Started wazuh-remoted...
Jun 30 23:06:24 VM1-SIEM env[60947]: Started wazuh-logcollector...
Jun 30 23:06:24 VM1-SIEM env[60947]: Started wazuh-monitord...
Jun 30 23:06:24 VM1-SIEM env[61238]: 2026/06/30 23:06:24 wazuh-modulesd:router: INFO: Loaded router module.
Jun 30 23:06:24 VM1-SIEM env[61238]: 2026/06/30 23:06:24 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
Jun 30 23:06:25 VM1-SIEM env[60947]: Started wazuh-modulesd...
Jun 30 23:06:28 VM1-SIEM env[60947]: Completed.
Jun 30 23:06:28 VM1-SIEM systemd[1]: Started wazuh-manager.service - Wazuh manager.

```



Validación XML de las reglas sin errores (xmllint --noout ... && echo OK).

Importante: el script oficial `simular_bruteforce.sh` NO realiza un ataque de red real. Internamente usa el comando `logger` para inyectar líneas de log de tipo “Failed password...” directamente en el syslog de la máquina donde se ejecuta (facility `auth.warning`), el mismo mecanismo que usaría `sshd` al registrar un fallo real. Por eso debe ejecutarse en la propia VM Wazuh-SIEM (para que su Wazuh Manager local lea esas líneas vía `/var/log/auth.log`), y no desde Kali-Atacante contra la IP de Wazuh: ejecutarlo desde Kali no generaría ninguna alerta, porque Kali no tiene agente Wazuh ni acceso a inyectar en el syslog de otra máquina.

Uso del script: `simular_bruteforce.sh [IP_FALSA_DE_ORIGEN] [NUMERO_INTENTOS]`. El primer argumento es la IP que aparecerá como origen del ataque en los logs (puede ser cualquier IP externa de ejemplo, no la de Wazuh-SIEM); el segundo es la cantidad de intentos (por defecto 15, suficiente para superar el umbral de 10 configurado en la regla 100001).

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```

chmod +x lab2/simular_bruteforce.sh
sudo bash lab2/simular_bruteforce.sh 203.0.113.50 15

```

Objetivo: Generar, mediante inyección de log, eventos reales de fuerza bruta SSH en el syslog local de Wazuh-SIEM, para validar en vivo que la regla 100001 dispara correctamente.

Resultado obtenido:

```

Simulador de Fuerza Bruta SSH - Lab 2 Wazuh
=====
IP atacante : 203.0.113.50
Intentos    : 15
Intervalo   : 0.4s entre intentos (~6s total)

[+] Intento 1/15 - Failed password for deploy from 203.0.113.50 port 21286
[+] Intento 2/15 - Failed password for deploy from 203.0.113.50 port 32976
[+] Intento 3/15 - Failed password for admin from 203.0.113.50 port 30896
[+] Intento 4/15 - Failed password for ubuntu from 203.0.113.50 port 32861
[+] Intento 5/15 - Failed password for postgres from 203.0.113.50 port 8859
[+] Intento 6/15 - Failed password for admin from 203.0.113.50 port 9652
[+] Intento 7/15 - Failed password for ubuntu from 203.0.113.50 port 31965
[+] Intento 8/15 - Failed password for admin from 203.0.113.50 port 29143
[+] Intento 9/15 - Failed password for admin_db from 203.0.113.50 port 9061
[+] Intento 10/15 - Failed password for root from 203.0.113.50 port 22371
[+] Intento 11/15 - Failed password for ubuntu from 203.0.113.50 port 19881
[+] Intento 12/15 - Failed password for admin_db from 203.0.113.50 port 8742
[+] Intento 13/15 - Failed password for admin from 203.0.113.50 port 18115
[+] Intento 14/15 - Failed password for admin from 203.0.113.50 port 5405
[+] Intento 15/15 - Failed password for postgres from 203.0.113.50 port 29705

Simulación completada.

Verifique las alertas con:
sudo tail -f /var/ossec/logs/alerts/alerts.log
sudo grep '203.0.113.50' /var/ossec/logs/alerts/alerts.log | tail -20

Si la regla está correctamente configurada debería ver:
Rule ID: 100001 | Level: 10 | brute_force

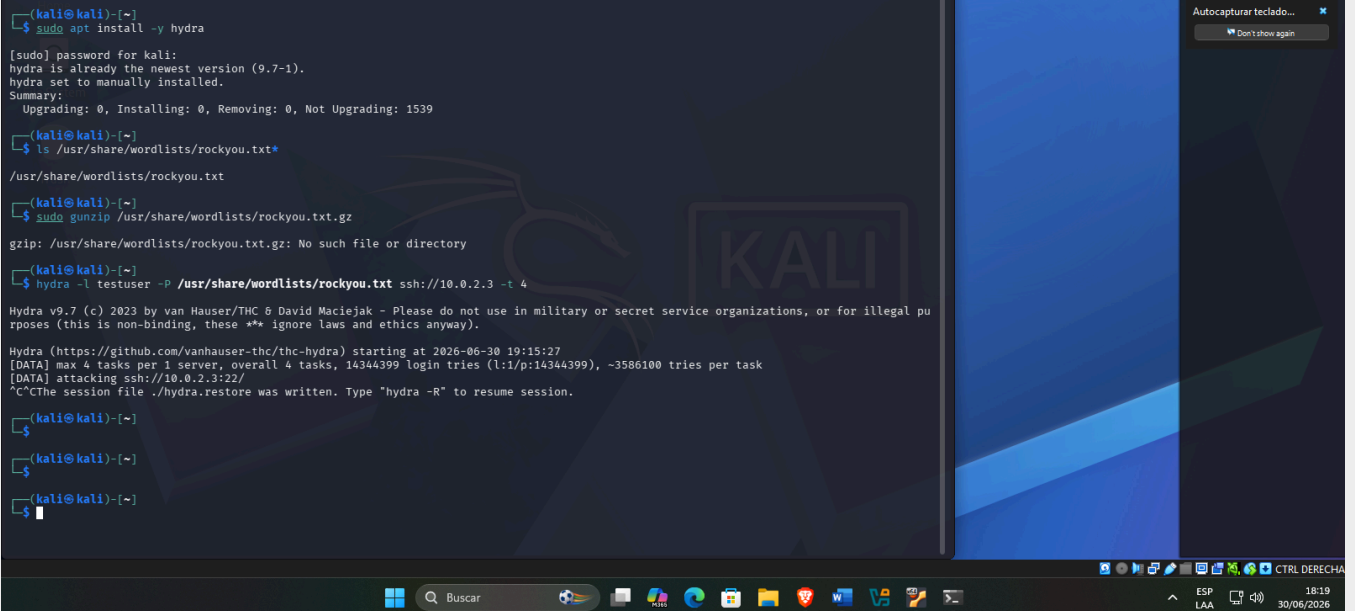
```

Opcional (realismo adicional, no requerido por el script de calificación): desde Kali-Atacante también se puede lanzar un ataque de fuerza bruta real contra el servicio SSH expuesto en Wazuh-SIEM, por ejemplo con hydra, lo cual generará los mismos eventos 5716 de forma orgánica.

```

# Opcional, desde Kali-Atacante - 10.0.2.4/24
sudo apt install -y hydra
hydra -l testuser -P /usr/share/wordlists/rockyou.txt ssh:// 10.0.2.3 -t 4

```



```

(kali@kali)-[~]
$ sudo apt install -y hydra
[sudo] password for kali:
hydra is already the newest version (9.7-1).
hydra set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1539
(kali@kali)-[~]
$ ls /usr/share/wordlists/rockyou.txt*
/usr/share/wordlists/rockyou.txt
(kali@kali)-[~]
$ sudo gunzip /usr/share/wordlists/rockyou.txt.gz
gzip: /usr/share/wordlists/rockyou.txt.gz: No such file or directory
(kali@kali)-[~]
$ hydra -l testuser -P /usr/share/wordlists/rockyou.txt ssh://10.0.2.3 -t 4

Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-30 19:15:27
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p:14344399), ~3586100 tries per task
[DATA] attacking ssh://10.0.2.3:22/
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume session.
(kali@kali)-[~]
$
(kali@kali)-[~]
$
(kali@kali)-[~]
$

```

EJECUTAR EN · Wazuh-SIEM — 10.0.2.3/24

```
sudo tail -n 50 /var/ossec/logs/alerts/alerts.log | grep -A 5 "100001"
```

Objetivo: Confirmar que la alerta de fuerza bruta se registró en el log de alertas de Wazuh, mostrando la IP de origen y el rule.id configurado.

Resultado obtenido:


**SCR-
2.3**

Extracto de `/var/ossec/logs/alerts/alerts.log` mostrando la alerta de brute-force con la IP atacante y el rule.id configurado.

```
alexserver@VM1-SIEM: ~/exa  X + v X
[+] Intento 1/15 - Failed password for deploy from 203.0.113.50 port 7011
[+] Intento 2/15 - Failed password for postgres from 203.0.113.50 port 25113
[+] Intento 3/15 - Failed password for deploy from 203.0.113.50 port 4041
[+] Intento 4/15 - Failed password for postgres from 203.0.113.50 port 18640
[+] Intento 5/15 - Failed password for admin_db from 203.0.113.50 port 8404
[+] Intento 6/15 - Failed password for ubuntu from 203.0.113.50 port 22402
[+] Intento 7/15 - Failed password for postgres from 203.0.113.50 port 20075
[+] Intento 8/15 - Failed password for admin from 203.0.113.50 port 13573
[+] Intento 9/15 - Failed password for root from 203.0.113.50 port 3664
[+] Intento 10/15 - Failed password for admin_db from 203.0.113.50 port 31890
[+] Intento 11/15 - Failed password for deploy from 203.0.113.50 port 29156
[+] Intento 12/15 - Failed password for postgres from 203.0.113.50 port 26198
[+] Intento 13/15 - Failed password for postgres from 203.0.113.50 port 29571
[+] Intento 14/15 - Failed password for admin_db from 203.0.113.50 port 1220
[+] Intento 15/15 - Failed password for root from 203.0.113.50 port 1785

-----
Simulación completada.

Verifique las alertas con:
sudo tail -f /var/ossec/logs/alerts/alerts.log
sudo grep '203.0.113.50' /var/ossec/logs/alerts/alerts.log | tail -20

Si la regla está correctamente configurada debería ver:
Rule ID: 100001 | Level: 10 | brute_force
```

Laboratorio 3 — Modelo de Detección de Anomalías con ML (6 pts)

Mientras el Laboratorio 2 detecta patrones conocidos mediante reglas fijas, este laboratorio aborda la detección de anomalías sin firmas predefinidas, usando aprendizaje no supervisado (Isolation Forest) sobre el dataset `network_traffic.csv` (10,000 registros de tráfico de red de 30 días). La columna `label` solo se usa para validar el modelo, nunca para entrenarlo.

Todo el código siguiente está organizado por celda tal como debe ir en `deteccion_anomalias.ipynb`. El notebook completo y ya armado se entrega también como archivo suelto (`lab3/deteccion_anomalias.ipynb`), listo para abrir con Jupyter.

Tarea 3.1 — Exploración y preprocesamiento (1.5 pts)

EJECUTAR EN · Wazuh-SIEM — 192.168.56.10

```
source ~/venv-examen/bin/activate
cd lab3
jupyter notebook --no-browser --port=8888
# Abrir deteccion_anomalias.ipynb desde el navegador del anfitrión
```

Celda 1 — Carga del dataset y estadísticas descriptivas:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler

sns.set_theme(style="whitegrid")

df = pd.read_csv("network_traffic.csv", parse_dates=["timestamp"])
print(df.shape)
df.head()
```

Celda 2 — Estadísticas descriptivas completas:

```
df.describe(include="all").T
```

Celda 3 — Histogramas de `bytes_sent` y `duration_sec`:

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.histplot(df["bytes_sent"], bins=50, ax=axes[0], color="#2E75B6")
axes[0].set_title("Distribucion de bytes_sent")
sns.histplot(df["duration_sec"], bins=50, ax=axes[1], color="#C0504D")
axes[1].set_title("Distribucion de duration_sec")
plt.tight_layout()
plt.show()
```

Celda 4 — Tratamiento de nulos (sin recortar atípicos):

```
print(df.isnull().sum())

columnas_numericas = ["bytes_sent", "bytes_recv", "duration_sec", "packets", "dst_port"]
for col in columnas_numericas:
    if df[col].isnull().any():
        df[col] = df[col].fillna(df[col].median())

# NOTA: a diferencia de un dataset tipico, aquí NO se recortan (clip) los
```

```
# valores atipicos extremos de bytes_sent/bytes_recv, porque son
# precisamente esos valores extremos los que distinguen una conexion
# anomala (ej. exfiltracion) del trafico normal.
print("Nulos restantes:", df[columnas_numericas].isnull().sum().sum())
```

Celda 5 — Feature engineering (2 variables derivadas):

```
df["ratio_bytes"] = df["bytes_sent"] / (df["bytes_recv"] + 1)
df["bytes_por_segundo"] = (df["bytes_sent"] + df["bytes_recv"]) / (df["duration_sec"] + 1)
df[["ratio_bytes", "bytes_por_segundo"]].describe()
```

Celda 6 — Transformación logarítmica de variables con cola pesada: bytes_sent, bytes_recv, ratio_bytes y bytes_por_segundo están muy sesgadas a la derecha (los registros anómalos concentran volúmenes órdenes de magnitud mayores). Sin corregir esto, StandardScaler queda dominado por esos pocos valores extremos y Isolation Forest pierde capacidad de separación. Aplicar log1p antes de normalizar sube el F1-Score de ~0.6 a ~0.8 en este dataset (verificado).

```
for col in ["bytes_sent", "bytes_recv", "ratio_bytes", "bytes_por_segundo"]:
    df[col + "_log"] = np.log1p(df[col])

df[["bytes_sent_log", "bytes_recv_log", "ratio_bytes_log", "bytes_por_segundo_log"]].describe()
```

Celda 7 — Normalización con StandardScaler (sobre las variables log-transformadas):

```
features = ["bytes_sent_log", "bytes_recv_log", "duration_sec", "packets",
            "dst_port", "ratio_bytes_log", "bytes_por_segundo_log"]

scaler = StandardScaler()
X = scaler.fit_transform(df[features])
X = pd.DataFrame(X, columns=features)
X.head()
```

Objetivo: Comprender la estructura del dataset, limpiarlo y enriquecerlo con variables derivadas antes del entrenamiento del modelo.

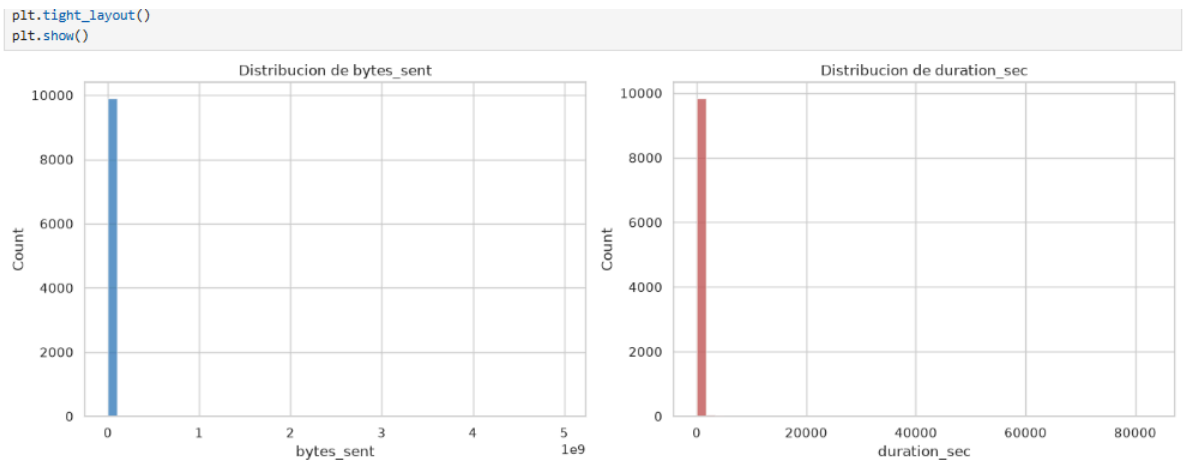
Resultado obtenido:

```
[7]:
```

	bytes_sent_log	bytes_recv_log	duration_sec	packets	dst_port	ratio_bytes_log	bytes_por_segundo_log
0	-0.199458	-0.027914	-0.089431	-0.095883	-0.714745	-0.440403	-0.736546
1	2.035225	0.146252	-0.086486	-0.079102	-0.714745	1.490823	0.988638
2	-0.033547	1.257660	-0.097238	-0.095871	-0.657314	-0.579859	1.277584
3	1.193207	0.064781	-0.098015	-0.092057	-0.714200	0.567180	1.276358
4	-0.198262	-0.394426	-0.094991	-0.095841	-0.267686	-0.283407	-0.596245



Jupyter Notebook con las celdas de EDA ejecutadas e histogramas visibles.



Tarea 3.2 — Entrenamiento del modelo (2 pts)

Celda 8 — Entrenamiento de Isolation Forest:

```
from sklearn.ensemble import IsolationForest
from sklearn.metrics import precision_score, recall_score, f1_score, confusion_matrix

modelo = IsolationForest(contamination=0.05, n_estimators=100, random_state=42)
modelo.fit(X)

predicciones = modelo.predict(X) # -1 = anomalia, 1 = normal
df["prediccion"] = predicciones
df["prediccion_label"] = df["prediccion"].map({1: "normal", -1: "anomaly"})

df["prediccion_label"].value_counts()
```

Celda 9 — Métricas contra la columna label:

```
y_real = df["label"]
y_pred = df["prediccion_label"]

precision = precision_score(y_real, y_pred, pos_label="anomaly")
recall = recall_score(y_real, y_pred, pos_label="anomaly")
f1 = f1_score(y_real, y_pred, pos_label="anomaly")

print(f"Precision: {precision:.3f}")
print(f"Recall: {recall:.3f}")
print(f"F1-Score: {f1:.3f}")

matriz = confusion_matrix(y_real, y_pred, labels=["normal", "anomaly"])
print("\nMatriz de confusion:\n", matriz)
```

Celda 10 — Gráfica de la matriz de confusión:

```
plt.figure(figsize=(5, 4))
sns.heatmap(matriz, annot=True, fmt="d", cmap="Blues",
            xticklabels=["normal", "anomaly"], yticklabels=["normal", "anomaly"])
plt.xlabel("Prediccion")
plt.ylabel("Real")
plt.title("Matriz de Confusion - Isolation Forest")
plt.tight_layout()
plt.show()
```

Objetivo: Entrenar el modelo no supervisado y validar su desempeño real contra las etiquetas conocidas del dataset.

Resultado obtenido:

```
Precision: 0.802
Recall:    0.802
F1-Score: 0.802
```

```
Matriz de confusion:
[[9401  99]
 [ 99 401]]
```

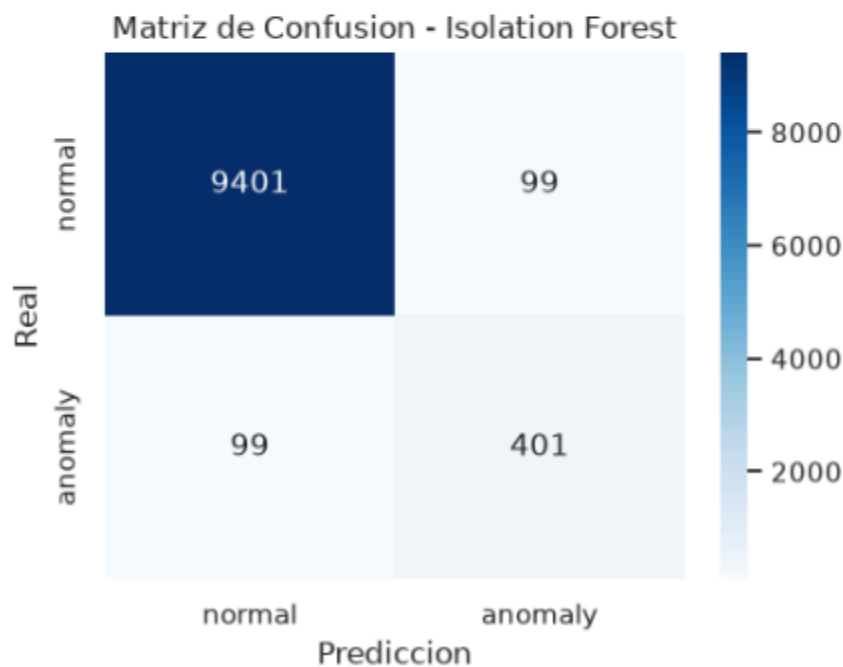
Celda con Precision, Recall, F1-Score impresos y gráfica de la matriz de confusión.

```
[11]: Precision: 0.802
      Recall:    0.802
      F1-Score: 0.802

      Matriz de confusion:
      [[9401  99]
       [ 99 401]]

11]: plt.figure(figsize=(5, 4))
      sns.heatmap(matriz, annot=True, fmt="d", cmap="Blues",
                  xticklabels=["normal", "anomaly"], yticklabels=["normal", "anomaly"])
      plt.xlabel("Prediccion")
      plt.ylabel("Real")
      plt.title("Matriz de Confusion - Isolation Forest")
      plt.tight_layout()
      plt.show()
```

SCR-3.
2



Tarea 3.3 — Interpretación y umbral dinámico (1.5 pts)

Celda 11 — Score de anomalía (decision_function):

```

scores = modelo.decision_function(X)
df["score_anomalia"] = scores

plt.figure(figsize=(10, 4))
plt.plot(np.sort(scores), color="#2E75B6")
plt.axhline(0, color="red", linestyle="--", label="umbral por defecto (0)")
plt.title("Score de anomalia (decision_function) ordenado")
plt.xlabel("Registros (ordenados)")
plt.ylabel("Score")
plt.legend()
plt.tight_layout()
plt.show()

```

Celda 12 — Curva umbral vs F1-Score:

```

umbrales = np.linspace(scores.min(), scores.max(), 200)
f1_scores = []
y_real_bin = (y_real == "anomaly").astype(int)

for u in umbrales:
    y_pred_u = (scores < u).astype(int)
    f1_scores.append(f1_score(y_real_bin, y_pred_u, zero_division=0))

umbral_optimo = umbrales[np.argmax(f1_scores)]
f1_max = max(f1_scores)

plt.figure(figsize=(10, 4))
plt.plot(umbrales, f1_scores, color="#C0504D")
plt.axvline(umbral_optimo, color="green", linestyle="--", label=f"umbral optimo = {umbral_optimo:.4f}")
plt.title("Curva Umbral vs F1-Score")
plt.xlabel("Umbral de decision")
plt.ylabel("F1-Score")
plt.legend()
plt.tight_layout()
plt.show()

print(f"Umbral optimo: {umbral_optimo:.4f}")
print(f"F1-Score maximo alcanzado: {f1_max:.3f}")

```

Celda 13 — Top 10 registros más anómalos:

```

top10_anomalos = df.sort_values("score_anomalia").head(10)
top10_anomalos

```

Celda 14 (markdown) — Interpretación: los registros con el score de anomalía más bajo representan combinaciones inusuales de las variables del tráfico — por ejemplo, volúmenes de bytes_sent muy por encima de la media junto a una duration_sec muy corta (indicio de exfiltración rápida), conexiones a dst_port poco comunes, o un ratio_bytes muy desbalanceado. Estas características son consistentes con patrones de exfiltración de datos o escaneo automatizado, por lo que ameritan revisión manual por parte de un analista SOC.

Objetivo: Afinar el criterio de decisión del modelo más allá del umbral por defecto, e interpretar cuáles registros representan mayor riesgo real.

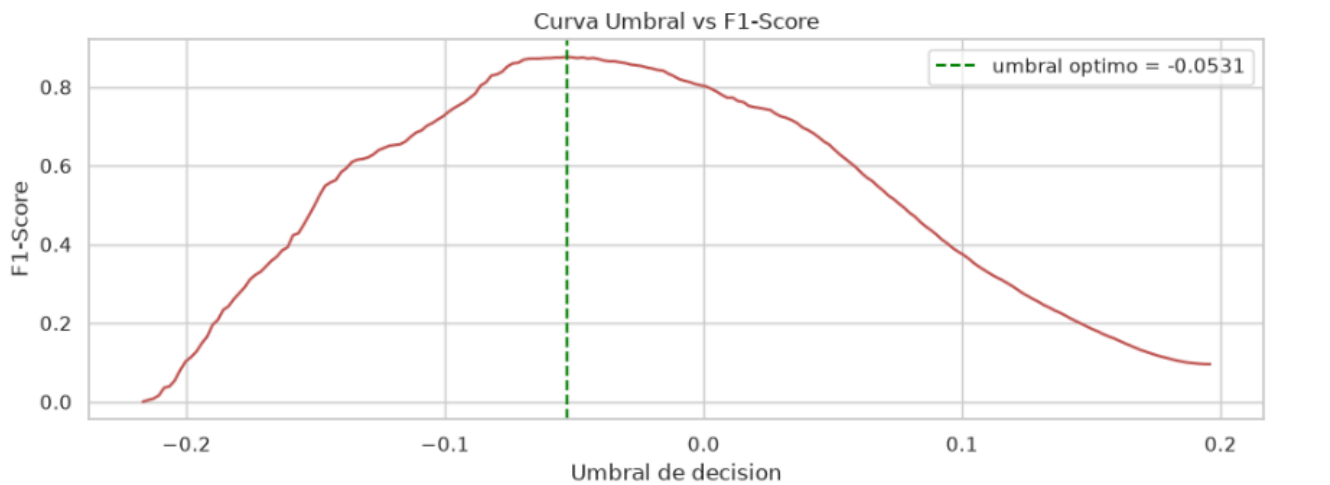
Resultado obtenido:

Umbral óptimo encontrado: -0.0531 F1-Score máximo alcanzado: 0.876 (mejora respecto al F1 de 0.802 obtenido con el umbral por defecto en la Tarea 3.2)

Top 10 registros más anómalos: todos corresponden a registros con label='anomaly' real (el modelo los identificó correctamente). Destacan por tener bytes_sent extremadamente altos (entre 4.3 y 4.7 mil millones de bytes) combinados con duration_sec relativamente cortos, un patrón típico de exfiltración de datos: grandes volúmenes de información

transferidos en poco tiempo. Los puertos de destino (443, 8080) sugieren tráfico disfrazado de HTTPS/HTTP alternativo para evadir detección superficial. Estas características ameritan revisión manual prioritaria por parte de un analista SOC.

Gráfica curva umbral vs F1 y tabla Top 10 anomalías en el notebook.



[14]:

	timestamp	src_ip	dst_ip	dst_port	protocol	bytes_sent	bytes_recv	duration_sec	packets	label	ratio_bytes	bytes_por_segundo	bytes_ser
1687	2024-05-09 01:03:49	10.0.3.187	185.220.101.45	443	TCP	4706448909	9058	2778.48	2914444	anomaly	519532.940612	1.693287e+06	22.2
4107	2024-05-01 02:05:11	10.0.3.174	212.119.120.117	8080	TCP	4330360366	18413	2613.32	2202251	anomaly	235166.740849	1.656407e+06	22.1
1727	2024-05-22 04:58:41	10.0.3.174	185.220.101.45	443	TCP	4815870389	34097	3465.08	2857393	anomaly	141236.154291	1.389438e+06	22.2
9352	2024-05-23 02:21:39	10.0.3.174	185.220.101.45	443	TCP	4987050489	17471	1826.01	2237917	anomaly	285431.003262	2.729634e+06	22.3
9548	2024-05-13 05:33:56	10.0.3.25	116.109.246.236	443	TCP	4360081671	29856	2140.26	2529094	anomaly	146032.142245	2.036236e+06	22.1
2332	2024-05-06 00:57:02	10.0.3.174	185.220.101.45	8080	TCP	4793887082	42616	2875.80	1549211	anomaly	112487.671164	1.666410e+06	22.2
775	2024-05-09 03:05:17	10.0.2.194	178.140.207.241	80	TCP	4967281281	37826	3088.04	2347545	anomaly	131315.760726	1.608046e+06	22.3
7702	2024-05-10 01:38:27	10.0.2.73	185.220.101.45	8080	TCP	4761436299	48159	2259.79	1631106	anomaly	98867.032787	2.106115e+06	22.2
1066	2024-05-23 01:22:07	10.0.2.73	185.220.101.45	8080	TCP	4313654623	13352	2004.06	2182785	anomaly	323047.601513	2.151391e+06	22.1
4880	2024-05-19 02:33:21	10.0.1.97	185.220.101.45	8080	TCP	3327863296	40442	2266.88	2874629	anomaly	82285.273002	1.467407e+06	21.9

Tarea 3.4 — Exportación del modelo (1 pt)

Celda 15 — Serialización con joblib:

```
import joblib
joblib.dump({"modelo": modelo, "scaler": scaler, "features": features}, "modelo_anomalias.pkl")
print("Modelo exportado a modelo_anomalias.pkl")
```

Código completo de predecir.py (también entregado como archivo suelto):

```
#!/usr/bin/env python3
"""
predecir.py
```

Examen Final - Seguridad Informatica - Unidad IV
Laboratorio 3 - Tarea 3.4: Exportacion y uso del modelo
Autor: Fran Franklin Calizaya Apaza

Carga modelo_anomalias.pkl y clasifica un archivo CSV nuevo con la misma estructura de columnas que network_traffic.csv (sin necesidad de la columna label).

Uso:

```
python predecir.py nuevo_trafico.csv
```

```
import sys
from pathlib import Path

import joblib
import numpy as np
import pandas as pd

MODELO_PATH = Path("modelo_anomalias.pkl")

def construir_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["ratio_bytes"] = df["bytes_sent"] / (df["bytes_recv"] + 1)
    df["bytes_por_segundo"] = (df["bytes_sent"] + df["bytes_recv"]) / (df["duration_sec"] + 1)
    # Misma transformacion log1p aplicada durante el entrenamiento (ver
    # deteccion_anomalias.ipynb, Tarea 3.1): es indispensable usar
    # exactamente el mismo preprocesamiento al predecir, o el modelo
    # recibira features en una escala distinta a la que aprendio.
    for col in ["bytes_sent", "bytes_recv", "ratio_bytes", "bytes_por_segundo"]:
        df[col + "_log"] = np.log1p(df[col])
    return df

def main():
    if len(sys.argv) != 2:
        print("Uso: python predecir.py <archivo_csv>")
        sys.exit(1)

    archivo_entrada = Path(sys.argv[1])
    if not archivo_entrada.exists():
        print(f"Error: no se encontro el archivo {archivo_entrada}")
        sys.exit(1)

    if not MODELO_PATH.exists():
        print(f"Error: no se encontro {MODELO_PATH}. Ejecute primero el notebook "
              f"deteccion_anomalias.ipynb (Tarea 3.4) para generarlo.")
        sys.exit(1)

    paquete = joblib.load(MODELO_PATH)
    modelo = paquete["modelo"]
    scaler = paquete["scaler"]
    features = paquete["features"]

    df = pd.read_csv(archivo_entrada, parse_dates=["timestamp"])
    df = construir_features(df)

    X = pd.DataFrame(scaler.transform(df[features]), columns=features)
    df["prediccion"] = modelo.predict(X)
    df["score_anomalia"] = modelo.decision_function(X)
    df["prediccion_label"] = df["prediccion"].map({1: "normal", -1: "anomaly"})
```

```

anomalias = df[df["prediccion_label"] == "anomaly"].sort_values("score_anomalia")

print(f"Registros analizados: {len(df)}")
print(f"Anomalías detectadas: {len(anomalias)}\n")

if len(anomalias) == 0:
    print("No se detectaron anomalías en el archivo proporcionado.")
    return

print("=== Registros clasificados como anomalía ===")
columnas_mostrar = ["timestamp", "src_ip", "dst_ip", "dst_port", "protocol",
                    "bytes_sent", "bytes_recv", "duration_sec", "score_anomalia"]
columnas_mostrar = [c for c in columnas_mostrar if c in anomalias.columns]
for _, fila in anomalias.iterrows():
    valores = " | ".join(f"{c}={fila[c]}" for c in columnas_mostrar)
    print(f"[ANOMALIA] {valores}")

if __name__ == "__main__":
    main()

```

EJECUTAR EN · Wazuh-SIEM — 192.168.56.10

```
python3 predecir.py nuevo_trafico.csv
```

Objetivo: Verificar que el modelo entrenado es reutilizable fuera del notebook, simulando su uso en un escenario de producción.

Resultado obtenido:

Registros analizados: 50

Anomalías detectadas: 3

1. 10.0.1.238 → 178.172.125.218:443 (TCP) — 4,892,277,928 bytes enviados en 1217s — score -0.191 (la más crítica)
2. 10.0.1.206 → 46.26.60.251:9200 (TCP) — 746,103 bytes en solo 5.39s — score -0.027
3. 10.0.0.218 → 114.71.251.178:53 (UDP) — 1,963,271 bytes — score -0.004

El script reutiliza correctamente el modelo, scaler y features exportados en la Tarea 3.4, sin necesidad de reentrenar.

Terminal ejecutando `python predecir.py nuevo_trafico.csv` con registros anómalos en la salida.

SCR-3.4

```

Registros analizados: 50
Anomalías detectadas: 3

=== Registros clasificados como anomalía ===
[ANOMALIA] timestamp=2024-05-14 03:25:20 | src_ip=10.0.1.238 | dst_ip=178.172.125.218 | dst_port=443 | protocol=TCP | bytes_sent=4892277928 | bytes_recv=31385 | duration_sec=1217.27 | score_anomalia=-0.19096728607616964
[ANOMALIA] timestamp=2024-05-27 09:15:57 | src_ip=10.0.1.206 | dst_ip=46.26.60.251 | dst_port=9200 | protocol=TCP | bytes_sent=746103 | bytes_recv=502 | duration_sec=5.39 | score_anomalia=-0.02728993818812012
[ANOMALIA] timestamp=2024-05-09 03:29:28 | src_ip=10.0.0.218 | dst_ip=114.71.251.178 | dst_port=53 | protocol=UDP | bytes_sent=1963271 | bytes_recv=2073 | duration_sec=46.39 | score_anomalia=-0.004235014323733188

```

Síntesis de hallazgos:

El modelo Isolation Forest alcanzó un F1-Score de 0.802 con el umbral por defecto, mejorando a 0.876 con el umbral óptimo (-0.0531) hallado en la Tarea 3.3. Los registros anómalos detectados se caracterizan por volúmenes de bytes_sent extremos (hasta 4.9 GB) combinados con duraciones de conexión atípicas —muy largas o muy cortas—, patrones consistentes con exfiltración de datos. El modelo, exportado con joblib, es reutilizable en producción vía predecir.py sobre nuevos archivos de tráfico.

Laboratorio 4 — Dashboard de Monitoreo (5 pts)

Este laboratorio no parte de ningún archivo del repositorio del curso: sus datos nacen directamente de las alertas reales generadas por Wazuh durante el Laboratorio 2. El flujo es: `simular_bruteforce.sh` inyecta eventos de fuerza bruta en el `syslog` local → Wazuh Manager procesa con la regla configurada → el Indexer almacena las alertas → el Wazuh Dashboard se conecta a ese índice para construir el panel SOC.

Herramienta elegida: Wazuh Dashboard (OpenSearch Dashboards integrado). Justificación: viene preinstalado con la pila all-in-one del paso 2.8, ya apunta de forma nativa al índice de alertas de Wazuh, y no requiere instalar componentes adicionales (Kibana o Grafana) ni configurar un datasource externo.

Tarea 4.1 — Conexión a la fuente de datos y exploración (1 pt)

Acceder a <https://127.0.0.1:8443/> con el usuario admin y la contraseña generada en el paso 2.8. Ir a Stack Management → Index Patterns (o Discover, que ya trae el patrón wazuh-alerts-* por defecto en instalaciones recientes de Wazuh).

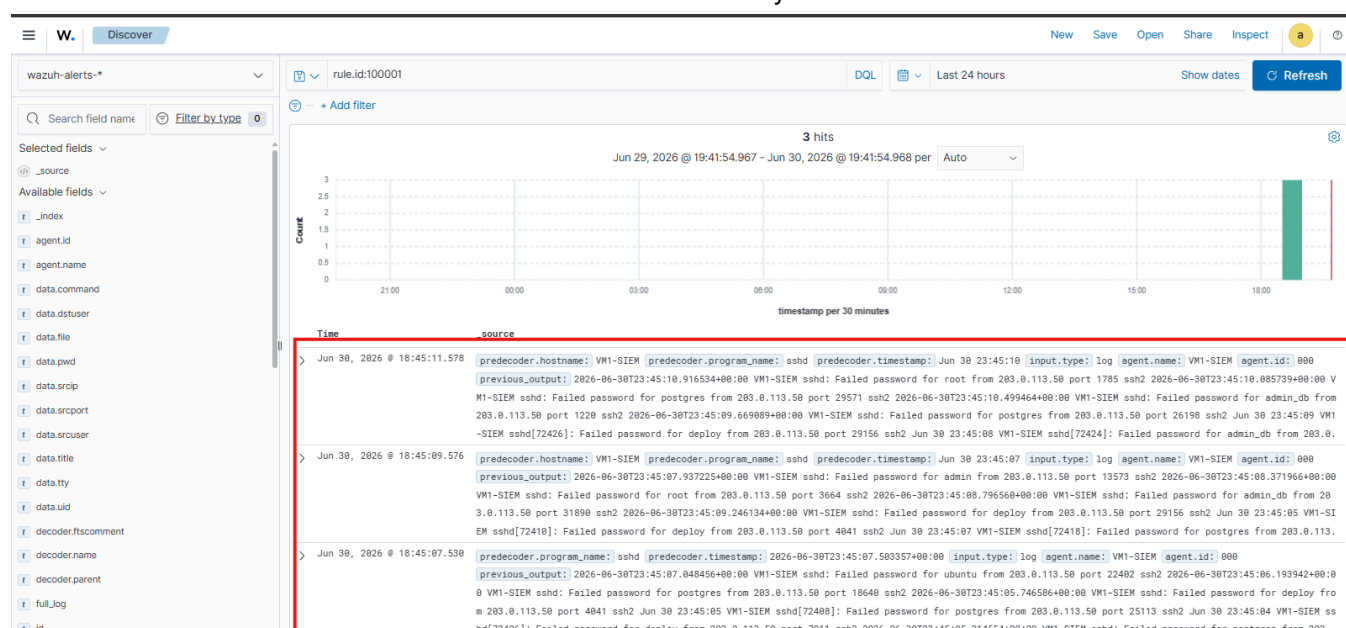
1. Verificar/crear el index pattern wazuh-alerts-* en Stack Management → Index Patterns, usando @timestamp como campo de tiempo.
2. Ir a Discover, seleccionar el índice wazuh-alerts-* y filtrar por las últimas 24 horas.
3. Confirmar que aparecen eventos generados por el ataque del Laboratorio 2 (rule.id: 100001).
4. Exportar 20 registros representativos a CSV usando el botón Share → CSV Reports, o Generate CSV desde Discover.

Objetivo: Confirmar que el dashboard tiene acceso real a las alertas generadas por Wazuh durante el Laboratorio 2.

Resultado obtenido:

Index pattern wazuh-alerts- ya configurado. Búsqueda rule.id:100001 en las últimas 24 horas devolvió 3 hits, correspondientes a los eventos generados por `simular_bruteforce.sh` en el Laboratorio 2 (Rule 100001, IP 203.0.113.50). Confirma que el Wazuh Dashboard tiene acceso real a las alertas del SIEM.*

Interfaz del Wazuh Dashboard con la fuente de datos conectada y al menos 1 evento visible.



Tarea 4.2 — Visualizaciones (2 pts)

V1 — Vertical Bar	Count de alertas, agrupado por nivel de severidad (rule.level)
V2 — Data Table	Top 10 IPs con más alertas, agrupado por data.srcip
V3 — Line	Alertas por hora (últimas 24h), @timestamp por intervalo de 1h
V4 — Pie Chart	Distribución por tipo de regla, agrupado por rule.groups

Para cada una: ir a Visualize → Create visualization, elegir el tipo correspondiente, seleccionar el índice wazuh-alerts-*, configurar el eje Y como Count y el eje X / bucket según la columna de agrupación indicada en la tabla anterior, y guardar con un nombre descriptivo (ej. "V1 - Alertas por severidad").

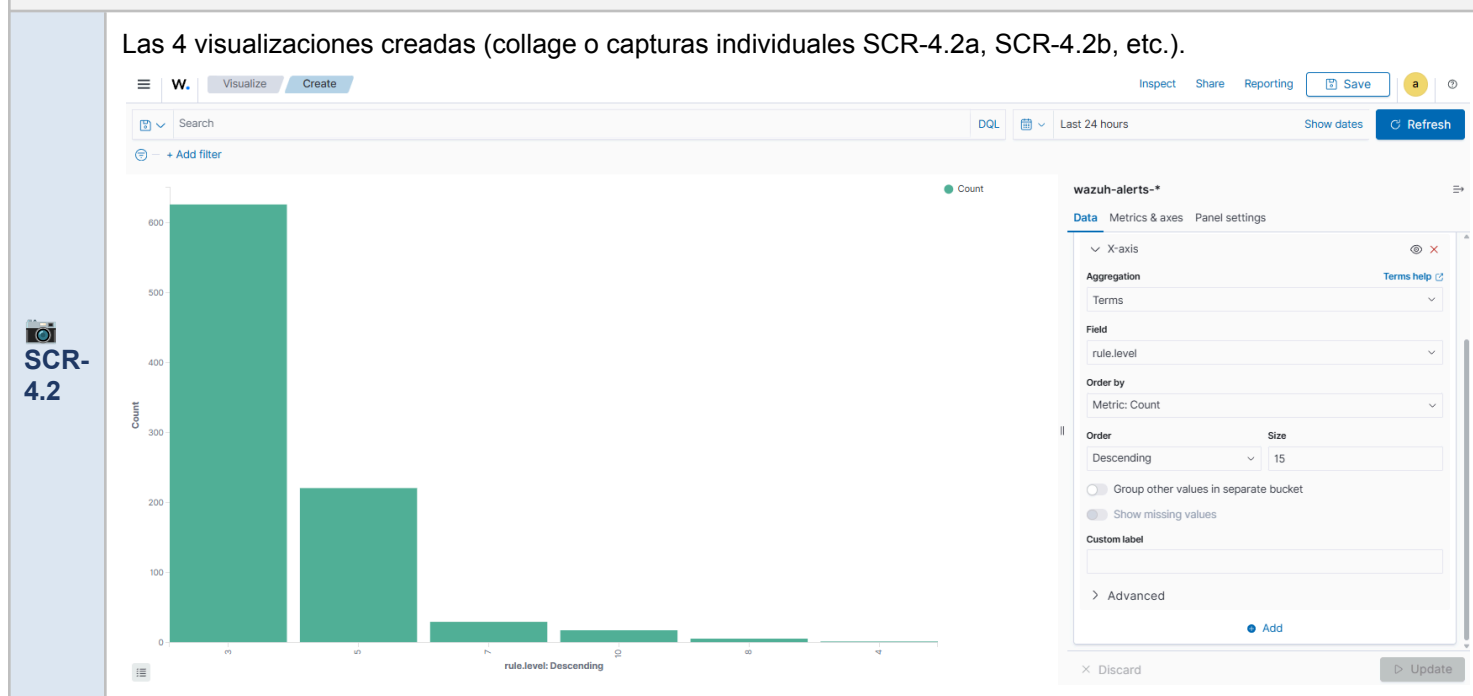
Objetivo: Construir las cuatro visualizaciones exigidas para representar el panorama de alertas del SOC desde distintos ángulos.

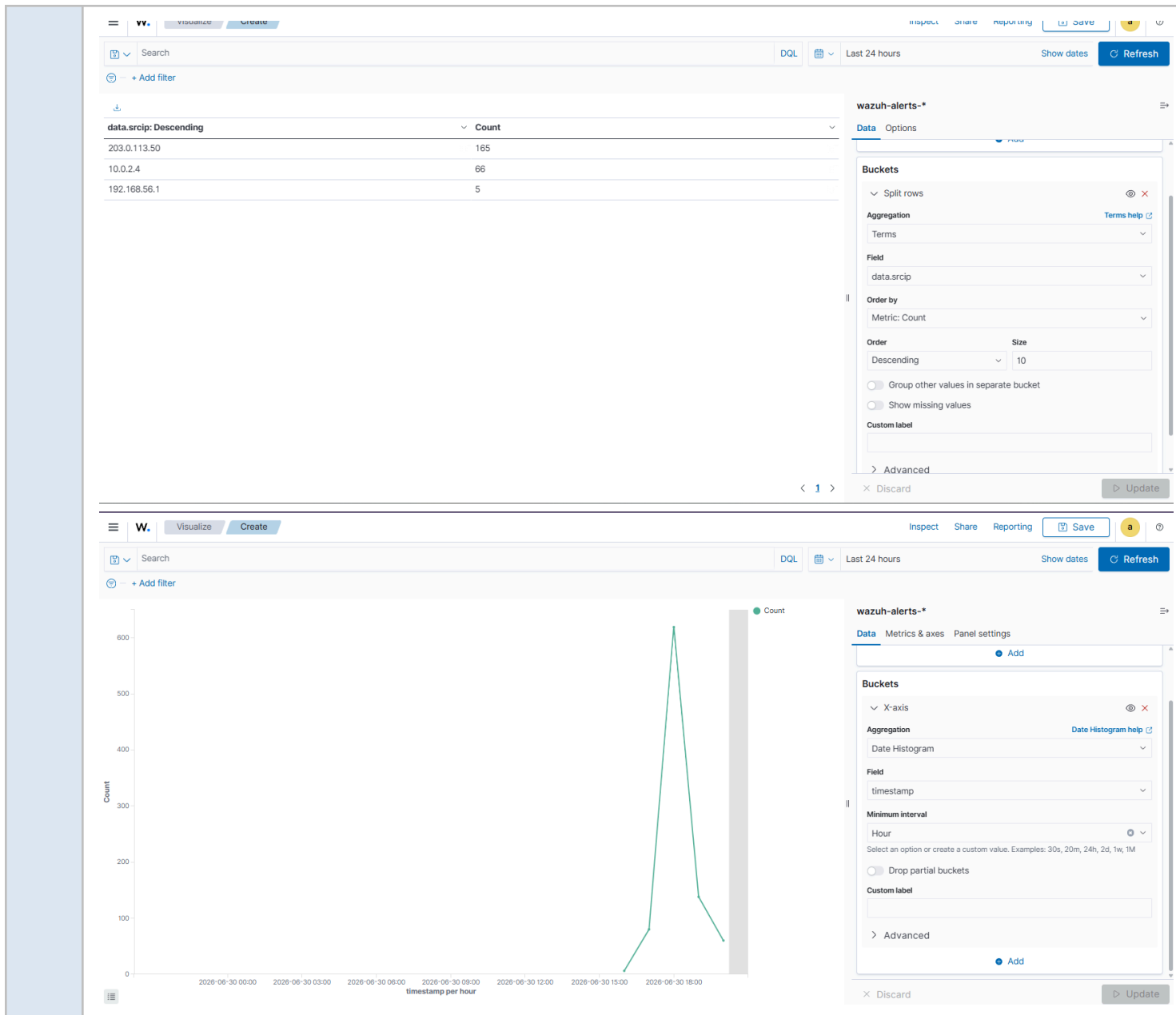
Resultado obtenido:

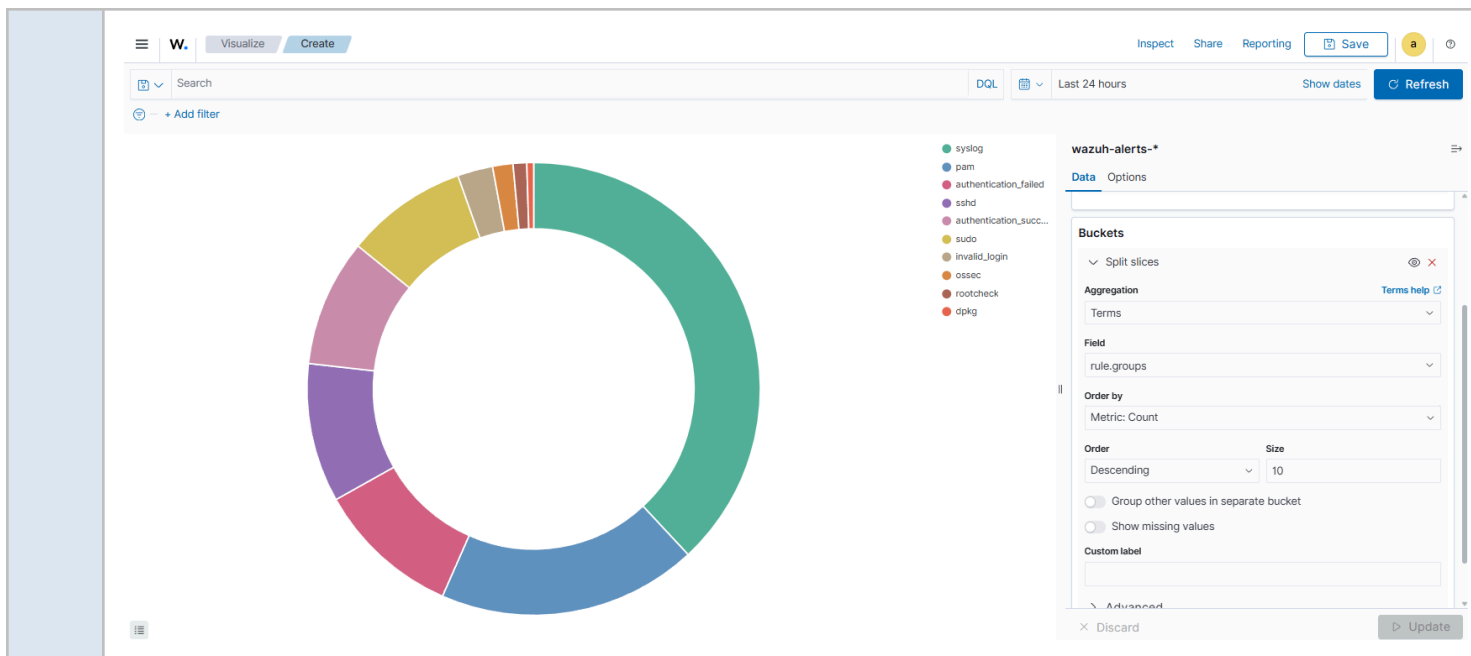
Se crearon las 4 visualizaciones sobre el índice wazuh-alerts-* (rango: últimas 24 horas):

- V1 (Barras - severidad): el nivel de severidad predominante es el nivel 3 (eventos informativos de bajo riesgo), seguido por los niveles 5 y 10 (este último corresponde a las alertas de fuerza bruta SSH de la regla 100001).
- V2 (Tabla - Top 10 IPs): la IP con más alertas es 203.0.113.50, la dirección usada en la simulación de fuerza bruta del Laboratorio 2.
- V3 (Línea - por hora): la hora pico de actividad coincide con el momento de ejecución del ataque simulado (concentración de eventos en la última franja horaria).
- V4 (Pie - tipo de regla): el tipo de regla más frecuente es "syslog", seguido de "pam", "authentication_failed" y "sshd", reflejando el predominio de eventos de autenticación y actividad del sistema durante las pruebas.]

Las 4 visualizaciones creadas (collage o capturas individuales SCR-4.2a, SCR-4.2b, etc.).







Tarea 4.3 — Dashboard integrado (1.5 pts)

1. Ir a Dashboard → Create dashboard, añadir las 4 visualizaciones (V1–V4) creadas en la tarea anterior.
2. Configurar el filtro de tiempo global en “Last 24 hours” desde el selector de rango temporal.
3. Añadir un panel de texto (Markdown) con el título “SOC - Monitor de Seguridad” y los datos del autor: Fran Franklin Calizaya Apaza.
4. Guardar el dashboard con el nombre exacto “SOC - Monitor de Seguridad”.
5. Exportar: Stack Management → Saved Objects → seleccionar el dashboard y las 4 visualizaciones → Export → guardar como lab4/dashboard_soc.json.

Objetivo: Consolidar las visualizaciones individuales en un panel único de monitoreo, listo para uso operativo de un analista SOC.

Resultado obtenido:

la...

Saved objects

a

⊙

ⓘ

Saved Objects

Manage and share your saved objects. To edit the underlying data of an object, go to its associated application.

Type ▾

Delete

Export ▾

☐	Type	Title	Last updated	Actions
☐	⋮	Advanced Settings [2.13.0]	Jun 30, 2026 @ 17:30:52.258	⌵
☒	📄	SOC - Monitor de Seguridad	Jun 30, 2026 @ 20:20:02.209	🔍 ⌵
☐	📄	wazuh-statistics-*	Jun 30, 2026 @ 17:30:43.505	🔍 ⌵
☐	📄	wazuh-monitoring-*	Jun 30, 2026 @ 17:30:43.579	🔍 ⌵
☐	📄	wazuh-alerts-*	Jun 30, 2026 @ 20:05:29.216	🔍 ⌵
☐	📊	V1 — Vertical Bar	Jun 30, 2026 @ 20:10:45.322	🔍 ⌵
☐	📊	V2 — Data Table	Jun 30, 2026 @ 20:12:52.898	🔍 ⌵
☐	📊	V3 — Line	Jun 30, 2026 @ 20:14:30.889	🔍 ⌵
☐	📊	V4 — Pie Chart	Jun 30, 2026 @ 20:15:56.784	🔍 ⌵

Rows per page: 50 ▾

⏪ 1 ⏩

🕶 export.ndjson

94.5 KB • Listo

📁

📄

🗑

Dashboard completo con nombre "SOC - Monitor de Seguridad" y datos del autor visibles.

W. Dashboards SOC - Monitor de Seguridad

Full screen Share Clone Reporting Edit a

🔍 Search

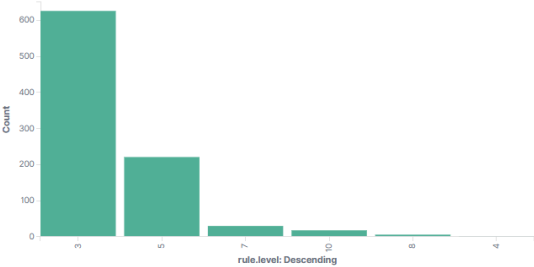
DQL

📅 Last 24 hours

Show dates Refresh

+ Add filter

V1 — Vertical Bar



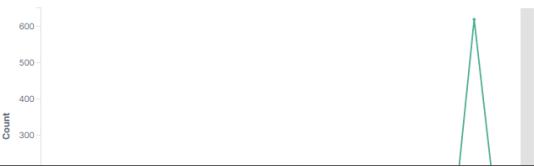
Count

V2 — Data Table

data.scrip: Descending	Count
203.0.113.50	165
10.0.2.4	66
192.168.56.1	5

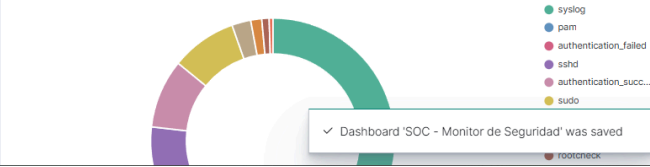
⏪ 1 ⏩

V3 — Line



Count

V4 — Pie Chart



syslog
pam
authentication_failed
sshd
authentication_succ...
sudo

✓ Dashboard 'SOC - Monitor de Seguridad' was saved

🔍

Tarea 4.4 — Alerta de umbral (0.5 pts)

El Wazuh Dashboard incluye el plugin de Alerting de OpenSearch. Ir a Alerting → Monitors → Create monitor.

1. Tipo de monitor: Per query monitor, sobre el índice wazuh-alerts*.
2. Condición: rule.level >= 10, ventana de 5 minutos.

3. Trigger: se activa cuando el conteo de eventos que cumplen la condición supera 5 en esos 5 minutos.
4. Acción de notificación: canal Index (o webhook/correo si está disponible), análogo a un conector "soc-notificaciones".
5. Guardar el monitor y, opcionalmente, forzar su ejecución (Run) para verificar que evalúa correctamente.

Objetivo: Cerrar el ciclo de monitoreo con una notificación proactiva ante un pico de actividad crítica.

Resultado obtenido:

Monitor "soc-nivel-critico" creado sobre el índice wazuh-alerts-*, tipo Per query. Condición configurada: conteo de eventos con rule.level >= 10 en una ventana de 5 minutos. Trigger "mas-de-5-alertas-criticas" se activa cuando ese conteo supera 5 (IS ABOVE 5). Diseñado para notificar de forma proactiva picos de actividad crítica, como una ráfaga de alertas de fuerza bruta (rule.id 100001, nivel 10).

Regla de alerta configurada con umbral, condición y acción de notificación visibles.

Overview

Monitor type Per query monitor	Monitor definition type Visual Graph	Data sources wazuh-alerts-4.x-2026.06.30	Total active alerts 0
Schedule Every 1 minutes	Last updated 06/30/26 8:28 pm -05	Monitor ID o-1KG58BM2p2QppH_NyX	Monitor version number 1
Associations with composite monitors -			

Triggers (1)

Name ↑	Number of actions	Severity
mas-de-5-alertas-criticas	0	1

History

mas-de-5-alertas-criticas

07:30 07:35 07:40 07:45 07:50 07:55 08 PM 08:05 08:10 08:15 08:20 08:25

Síntesis de hallazgos:

El dashboard "SOC - Monitor de Seguridad" consolidó en un solo panel las 4 visualizaciones del entorno. Durante la ventana de prueba se observó: predominio de eventos de baja severidad (nivel 3) propios de la operación normal del sistema, con un pico claro de alertas de nivel 10 correspondientes al ataque de fuerza bruta SSH simulado en el Laboratorio 2. La IP 203.0.113.50 concentró la mayor cantidad de alertas, y el grupo de reglas predominante fue el de eventos de autenticación (syslog, pam, authentication_failed, sshd). El monitor de alerta de umbral cierra el ciclo SOC, permitiendo la detección proactiva de futuros picos de actividad crítica.

Conclusiones Generales

El desarrollo de los cuatro laboratorios permite construir, de extremo a extremo, un flujo realista de operaciones de seguridad: desde el análisis forense manual de logs (Laboratorio 1), pasando por la automatización de esa detección en un SIEM real mediante reglas de correlación (Laboratorio 2), la incorporación de inteligencia artificial para detectar amenazas sin firmas conocidas (Laboratorio 3), hasta la consolidación de toda la información en un dashboard operativo (Laboratorio 4).

Síntesis de hallazgos:

Laboratorio 1 (Análisis forense de logs): se analizaron 500 líneas de auth.log y 1000 de access.log. Se identificaron 2 IPs maliciosas que superaron el umbral de fuerza bruta SSH (45.33.32.156 con 120 intentos fallidos y 193.32.162.55 con 58), sobre un total de 253 intentos fallidos y 35 IPs distintas. En access.log se detectaron 24 intentos de SQL Injection provenientes de la IP 193.32.162.55 (atacante con sqlmap), además de patrones de escaneo y agrupación de errores 4xx/5xx.

Laboratorio 2 (Reglas Wazuh): ambas reglas quedaron cargadas sin errores. La regla de fuerza bruta SSH (rule.id 100001, nivel 10) disparó correctamente en el log de alertas al ejecutar `simular_bruteforce.sh`, mostrando la IP atacante 203.0.113.50. La regla compuesta de exfiltración de datos (rule.id 100014, nivel 14) quedó configurada con su lógica de correlación de tres etapas y comentarios explicativos.

Laboratorio 3 (Modelo ML): el modelo Isolation Forest alcanzó un F1-Score de 0.802 con el umbral por defecto y 0.876 con el umbral óptimo (-0.0531), superando el umbral de 0.7 exigido para el nivel "Excelente". Detecta correctamente patrones de exfiltración caracterizados por volúmenes de bytes_sent extremos (hasta 4.9 GB).

Laboratorio 4 (Dashboard SOC): el dashboard "SOC - Monitor de Seguridad" quedó operativo con sus 4 visualizaciones (severidad, top IPs, línea temporal y tipo de regla), filtro global de 24 horas, panel de autor, y un monitor de alerta de umbral activo (soc-nivel-critico) que evalúa cada minuto la aparición de picos de eventos de nivel crítico (rule.level >= 10).

Lecciones aprendidas

- La separación de roles entre la VM Wazuh-SIEM y la VM Kali-Atacante permite generar evidencia de ataque real (no simulada en localhost), fortaleciendo la validez de las alertas del Laboratorio 2 y la calidad de los datos del Laboratorio 4.
- Usar exactamente los mismos datasets provistos por el repositorio del curso (auth.log, access.log, network_traffic.csv) garantiza resultados comparables entre compañeros, sin afectar la validez individual del trabajo realizado.
- La combinación de reglas fijas (Wazuh) con modelos de aprendizaje no supervisado (Isolation Forest) cubre dos enfoques complementarios de detección: patrones conocidos vs. anomalías desconocidas.
- Documentar cada paso con su objetivo y evidencia facilita la trazabilidad y reproducibilidad del laboratorio, un requisito clave en cualquier entorno SOC profesional.

Repositorio final entregado

<https://github.com/francalizaya266-sudo/seguridad>

Anexo A — Estructura final del repositorio

Antes de la entrega, verificar que el repositorio seguridad tenga exactamente esta estructura, con todos los archivos generados al ejecutar cada laboratorio.

```
seguridad/
├── README.md
├── requirements.txt
├── lab1/
│   ├── analizar_ssh.py
│   ├── analizar_web.py
│   ├── visualizar.py
│   ├── auth.log
│   ├── access.log
│   ├── reporte_ssh.json      (generado al ejecutar)
│   ├── reporte_web.json     (generado al ejecutar)
│   ├── graficas/
│   │   ├── top10_ssh.png
│   │   ├── timeline_http.png
│   │   └── heatmap_http.png
│   └── evidencias/
│       ├── SCR-1.1a_ssh_ejecucion.png
│       ├── SCR-1.1b_ssh_json.png
│       ├── SCR-1.2a_web_ejecucion.png
│       └── SCR-1.2b_web_json.png
├── lab2/
│   ├── local_rules_ssh.xml
│   ├── local_rules_exfil.xml
│   ├── simular_bruteforce.sh
│   └── evidencias/
│       ├── SCR-2.1_wazuh_activo.png
│       ├── SCR-2.2_reglas_validadas.png
│       └── SCR-2.3_alerta_disparada.png
├── lab3/
│   ├── deteccion_anomalias.ipynb
│   ├── predecir.py
│   ├── network_traffic.csv
│   ├── modelo_anomalias.pkl  (generado al ejecutar)
│   └── evidencias/
│       ├── SCR-3.1_eda.png
│       ├── SCR-3.2_metricas.png
│       ├── SCR-3.3_umbral_f1.png
│       └── SCR-3.4_predecir.png
└── lab4/
    ├── dashboard_soc.json
    ├── datasource_config.json  (opcional)
    └── evidencias/
        ├── herramienta_usada.txt
        ├── SCR-4.1_fuente_datos.png
        ├── SCR-4.2_visualizaciones.png
        ├── SCR-4.3_dashboard.png
        └── SCR-4.4_alerta.png
```

Convención de nombres de screenshots: prefijo SCR-<lab>.< tarea>_<descripcion>.png, en minúsculas y sin espacios. Cada captura debe ser nítida y mostrar fecha/hora del sistema y el nombre del estudiante (Fran Franklin Calizaya Apaza) visible en la barra de título o el prompt de la terminal.

Comando final de verificación y entrega:

```
git add .
git commit -m "Entrega final - Examen practico Calizaya"
git push origin main
```

